

多种情形下的种植优化策略研究

摘 要

随着乡村有机种植产业的发展，研究农作物的优化种植策略问题对乡村经济的可持续发展具有重要的现实意义。本文基于题目所给数据信息，建立优化种植策略模型分析求解，为促进有机种植产业发展提供了有效方案。

在进行问题分析求解前，需要对附件 1 和 2 的数据进行连接，并进行必要的数据整合与数据分析，初步了解该乡种植业的基本情况。

针对问题一，本文建立了基于**混合整数线性规划**模型的稳定情形最优种植方案。首先，进行数据预处理，将农作物数据集按作物类别和地块编号分为三类。接着，引入 4 种滞销因子：0%，25%，50%，75%考虑滞销情况，设计产量超出预期销售量部分的滞销浪费及降价 50%出售 2 种情况，共 8 种情景进行后续求解。随后，以是否在某地块种植某作物、作物种植面积为决策变量，以最大种植面积限制、不能连续重茬种植等农作物生长条件为约束，以最大化利润为目标函数，建立最优规划策略模型。对于该模型，我们利用 **GLPK 求解器**进行**线性规划**求解。最后，在滞销率为 50%的基础上，得出 2024-2030 年间，不同滞销和不同售卖情形下的最优种植策略。求得滞销浪费情景的最大合计利润为 68201176 元，降价出售的最大合计利润为 97112198 元。

针对问题二，在问题一的基础上建立波动情形下的种植优化策略模型。首先，为了引入市场和农业的波动变化与趋势效应，本文使用**蒙特卡洛方法**进行随机数值模拟，以满足各种农作物相关数据的不确定性。其次，针对农作物种植的潜在风险，本文引入**灾害因子**来度量自然灾害对农作物产量以及成本的影响程度，重新建立目标函数。最后，基于上述不确定性和潜在风险，我们在第一问的约束下使用 **GLPK 求解器**计算得出波动情形下最优的种植策略与最大利润。

针对问题三，在问题二的基础上建立现实情形下的优化种植模型。本文从市场和农业角度出发，分别利用**交叉价格弹性系数**和**余弦相似度**考虑农作物之间的替代性与互补性。同时，基于 **SEM 结构方程模型**分析了预期销售量与销售价格、种植成本之间的相关性，构建路径方程。基于上述分析，定义土地系数量化农作物之间的替代性与互补性，定义利润系数量化农作物的市场经济表现，对农作物进行综合评价。依据综合评价的 K-means 聚类结果，筛选出 5 种最优农作物。最后，将最优农作物数据带入问题二中的种植优化策略模型，同样使用 **GLPK 求解器**进行求解。将问题三的结果与问题二的结果进行比较，得出结论：第三问的种植策略优于第二问的种植策略。

综上所述，本文依据各题所给条件较全面地分析了农作物的种植策略，并引入了灾害因子满足各农作物的潜在风险性，提高了数据的有效性。最后我们给出了相应的最优种植策略。经过分析与验证，本文的模型具有合理性和一定的现实意义。

关键词：混合整数线性规划 蒙特卡洛 交叉弹性系数 余弦相似度 SEM 结构模型

一、 问题重述

1.1 问题背景

为了实现乡村经济的可持续发展，充分利用有限的耕地资源，发展因地制宜的有机种植产业有重大意义。

华北山区一乡村，受限于低温与有限的耕地资源，面临发展挑战。该乡现有 1201 亩露天耕地，分散为 34 块不同大小、4 种类型的地块。另有 20 个（16 个普通+4 个智慧）大棚，各棚 0.6 亩。露天耕地中，平旱地、梯田、山坡地适合单季粮食种植，水浇地则宜水稻或双季蔬菜。大棚配置上，普通大棚适种一季蔬菜与食用菌，智慧大棚则能双季蔬菜轮作。

为促进该乡种植业发展，需精心规划种植策略。确保作物不连作、防减产，且三年内每地块至少种一次豆类以优化土壤。同时，考虑耕种便捷与田间管理效率，作物布局需集中适度，避免地块细碎化种植。根据上述情况，可基于当前 2023 年作物种植情况及统计数据，制定科学合理的种植方案。

1.2 问题重述

问题设置是层层递进，并且服务于同一主题的，即综合考虑多种因素，优化该乡的农作物种植策略。

问题一：假设未来各作物预期销售量、成本、亩产量及价格稳定，当季种植即售。若产量超出预期销售，则超量部分无法正常售出。针对此情境，本问需分别为以下两种情况制定 2024 至 2030 年的最优农作物种植方案：

(1) 超出部分滞销，造成浪费。

(2) 超出部分按 2023 年销售价格的 50% 降价出售。

问题二：根据经验对各种作物的销售量与价格波动进行设定。本问需在第一问基础上，综合考虑这些不确定性和潜在风险，制定该乡村 2024 至 2030 年的最优农作物种植方案。

问题三：首先，考虑农作物间的可替代性和互补性，以及预期销售量、销售价格与种植成本的相关性。接着，在问题 2 的基础上，进一步融合这些复杂因素。通过模拟数据分析，制定该乡 2024 至 2030 年的最优农作物种植策略。最后，将该最优策略与问题二的结果进行比较分析。

二、 问题分析

2.1 问题一的分析

问题一需要制定稳定情形下的最优种植方案。首先，整理题中所给数据，根据附件 1 得到该乡土地面积、土地类型等相关数据；根据附件 2 得到该乡农作物亩产量、销售单价等相关数据，建立附件之间的自然连接。随后，假定各种农作物未来的预期销售量、种植成本、亩产量和销售价格相对于 2023 年保持稳定，每季种植的农作物在当季销售。如果某种作物每季的总产量超过相应的预期销售量，超过部分不能正常销售。题中设定情景如下：

(1) 超过部分滞销，造成浪费；

(2) 超过部分按 2023 年销售价格的 50% 降价出售。

鉴于滞销情况未知，本文设置滞销系数为 0%，25%，50%，75% 的四种情形[7]。根据不同情景建立不同的目标函数，即种植利润最大化。最理想的情况是没有滞销部分，此

时滞销系数为 0%。

最后，根据最大种植面积、不能连续重茬等约束条件建立混合整数线性规划模型，使用 GLPK 求解器得到该乡 2024~2030 年的最优农作物种植方案和最大利润。

2.2 问题二的分析

在第一问的基础上，第二问需要根据农作物的属性以及市场情况的变化趋势、波动范围重新求解最优种植策略。首先，针对种植业发展的不确定性，本文基于蒙特卡洛模拟结合置信区间预测未来七年农作物的相关数据。同时，针对种植业发展的潜在风险，引入灾害因子模拟自然灾害对农作物产量的影响[1]，重新制定约束条件和目标函数，建立混合整数线性规划模型。最后，在第一问的滞销系数情景下，使用 GLPK 求解器得到该乡 2024~2030 年的最优农作物种植方案和最大利润。

2.3 问题三的分析

在第二问的基础上，第三问需要综合考虑现实情况：

(1) 各种农作物之间可能存在一定的可替代性和互补性。

(2) 预期销售量与销售价格、种植成本之间存在一定的相关性。

首先，利用余弦相似度和交叉价格弹性系数分别从农业、市场角度考虑农作物的可替代性和互补性[2][7]。同时，建立 SEM 结构方程模型探究预期销售量与销售价格、种植成本之间的相关性，并基于因子载荷系数建立不同变量对预期销售量影响大小的回归模型。接着，基于上述相关性、替代性与互补性分析，引入土地系数与利润系数对农作物进行综合评价，并使用 K-means 算法对作物进行聚类分析，选取最具代表性的 5 种最优作物参与种植决策。最后，将最优作物带入第二问进行求解并对比第二问和第三问的优化结果。

综上，我们建立了三种优化模型：稳定情形下的优化种植模型、波动情形下的优化种植模型、现实情形下的优化种植模型，在不同的情形下，添加不同的条件，考虑不同的因素进行 MIP 规划，并利用 GLPK 求解器进行求解，得出最优的种植方案。

文章总体思路图如图 1 所示：

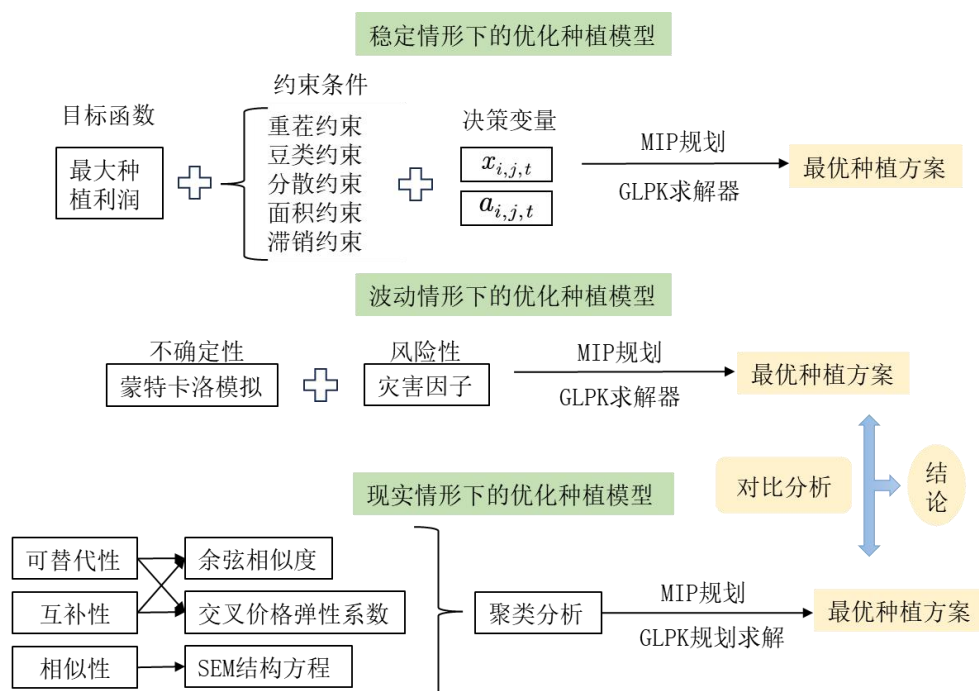


图 1 论文总体思路图

三、基本假设与符号说明

3.1 基本假设

1. 在优化种植模型中，以销售范围内的中位数作为基础销售单价。

原因：由于中位数不受极端值（异常值）的影响，中位数可以更好地反映数据的中心位置。

2. 在规划的 7 年时间内，农作物的加工运输等渠道不发生强烈变化。

原因：农作物的加工运输等是农产品销售的关键环节，对利润有很大程度上的影响，因此应当假设在规划期间农作物的加工运输等渠道不发生强烈变化。

3. 在规划期间，地块面积不发生变化，且不会有新土地投入使用。

原因：在规划期间为了保证数据的可用性，应当假设华北地区地块面积不发生变化。

4. 乡村可以应对各种灾害影响。

原因：为了保证土地作物种植的合理性，假设乡村可以正常应对灾害。

5. 在规划期间，政府对农户的补贴以及相关农业种植政策保持稳定。

原因：政府对农户的补贴特别是农业种植政策将会对农户的收成产生很大的影响，因此，为了保证合理性，应当假设规划期间的政府补贴及政策保持稳定。

3.2 符号说明

符号	说明	单位
$x_{i,j,t}$	第 i 个地块在 j 季节种植第 t 种作物的面积	亩
$a_{i,j,t}$	第 i 个地块在第 j 季节是否种植第 t 种作物（1 为是，0 为否）	无
S_t	第 t 种作物的预期销售量	吨
P_t	第 t 种作物的销售价格	吨
Y_t	第 t 种作物的亩产量	吨/亩
C_t	第 t 种作物的种植成本	元/亩
A_i	第 i 个地块的实际面积	亩

四、数据侧写

4.1.1 数据整合

考虑到本题两个附件数据之间存在关联关系，需在进行问题分析和解答前进行一定的数据整合。根据题目要求，综合附件 1、2 数据和信息，通过 Excel 数据透视表等方法对数据进行整合与分析。数据整合包括附件 1 和 2 的连接，依照作物、土地、季节名称等进行聚合；数据分析包括度季节的分析、地块类型的分析、作物种类的分析。

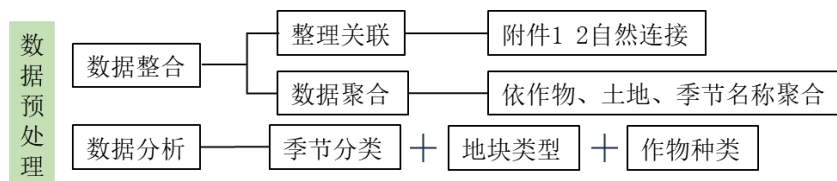
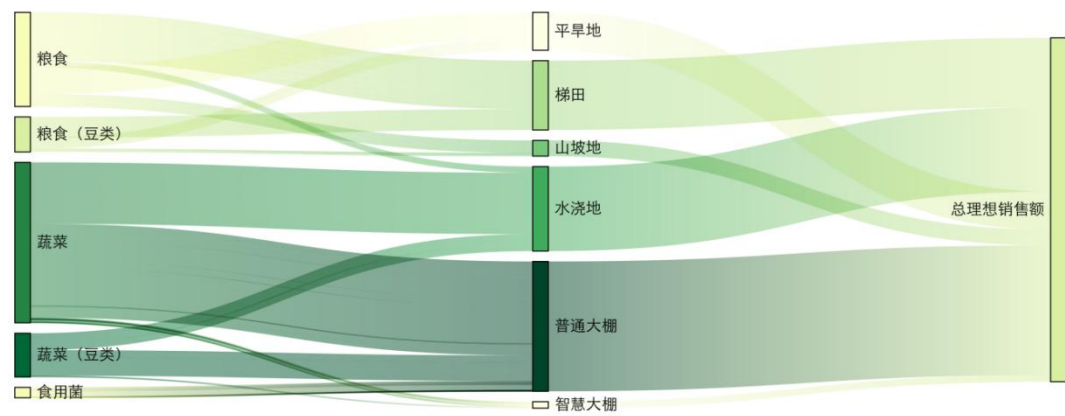


图 2 数据预处理过程

4.1.2 数据可视化分析

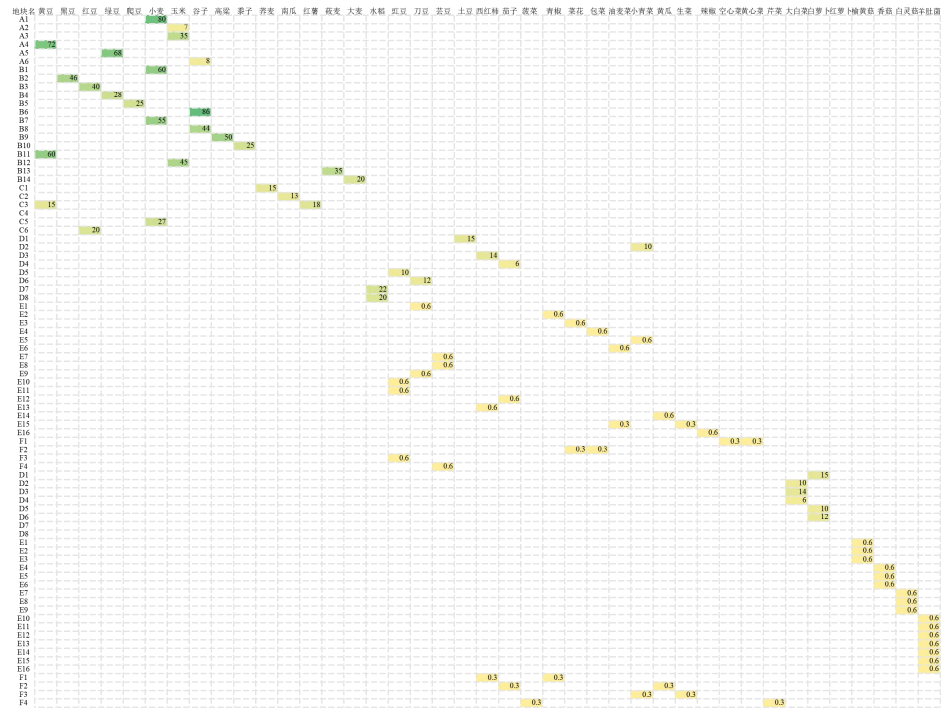
该乡有四种露天土地类型，两种大棚类型。露天土地中，平旱地、梯田、山坡地虽只能种植单季粮食作物，但占地 1201 亩，占总种植地块的 90%；两种大棚占地 12 亩，

可以种植两季蔬菜和食用菌，具有较高的生产效率和利润空间。



绘制 2023 年农作物种植与耕地类型的对应结构桑基图（完整图片参见附录）。从产量角度来看，粮食类作物和蔬菜类作物构成了该地最主要的农作物种类。其次，普通大棚和智慧大棚的产量较低，这与其占地面积有较大关系。从种植成本角度来看，四种露天耕地的种植成本占比最高，主要贡献作物为粮食类。从总理想销售额来看，普通大棚以较低的成本，较低的产量，贡献了最多的销售额，这与大棚的主要的作物类型和双季种植优势有较大关系。由此我们可以初步了解该地的土地结构与作物种植情况。

根据 2023 年种植情况数据，可视化每块耕地的作物种类、种植亩数与整体的作物种植分布情况如下图。横向为作物类型，纵向为两个季度与地块编号。其中高亮显示的表示在该地块种植该种作物，颜色越深表示种植面积越大。该分布图可以使我们对不同年份种植策略的差异性有更深入的了解。后续优化模型均可在此分布图中展示不同的种植方案。



五、 问题一模型的建立与求解

5.1 稳定情形下的优化种植模型

5.1.1 数据预处理

由于预期销售量未知，简单地把 2023 年的产量作为预期销售量可能与实际并不相符。因此，本文参考文献，对 2023 年的销售量进行分级假设，设计滞销系数为 25%，50%，75%，100%，作物的滞销量为总产量乘以滞销系数。本文设计在不同的滞销情景下求解两种滞销处理情景的最大收益与种植策略。

本题的决策变量同时包含 0-1 整数型变量 $a_{i,j,k}$ ，即是否种植某种作物和连续型变量 $x_{i,j,k}$ ，即某种作物在某块耕地上的种植面积。因此需要建立混合整数规划模型（MIP）进行求解。

考虑到题目中约束较多，作物类型和土地类型比较丰富，本文把数据按照 41 种作物类型和 6 种土地类型进行整合，得出三类数据：

第一类：在 A、B、C 土地上种植的粮食类单季农作物；

第二类：在 D、E、F 土地上种植的蔬菜类第一季农作物（该类包含水稻）；

第三类：在 D、E、F 土地上种植的蔬菜类和食用菌类第二季农作物。

通过对数据的分层，可以加入较完整的约束，减少数据标签的冲突性，提高数据利用的可行性。基于处理后的数据建立最优种植规划模型。

5.1.2 决策变量的确定

记第 i 个地块在第 j 季节种植第 t 种作物的面积（单位：亩）为 $x_{i,j,t}$ ；记 $a_{i,j,t}$ 第 i 个地块在第 j 季节是否种植第 t 种作物（1 为是，0 为否）。

5.1.3 目标函数的确定

本题设定了两种销售情景：产量超出预期销售量部分滞销；产量超出预期销售量部分半价出售。从种植策略制定者角度出发，农民更希望保证在总种植成本较低的前提下，获得较大收益。本文以 2023 年为基准年份，在农业市场稳定的情形下，考虑农业种植的实际情况，针对两种情景建立目标函数：

• 滞销浪费

考虑到超过预期销售量部分不能销售，需要对每种作物的收益取产量和预期销售量的最小值，总收益为各作物的收益减去种植成本。目标即为最大化该乡在 2024 年-2030 年的总收益：

$$\text{Max} \sum_t \min \left(\sum_{i,j} x_{i,j,t} \cdot Y_t, S_t \right) \cdot P_t - x_{i,j,t} \cdot C_t \quad (1)$$

其中，目标函数的符号已在符号说明中给出。

• 降价出售

此时超出预期销售量部分按 2023 年销售价格的 50% 降价出售，需要将每种作物的收益计算分为两部分：低于预期销售额的部分按全价出售，超过部分按半价出售。重新制定最大化总收益的目标函数：

$$\text{Max} \sum \left(\min \left(\sum_{i,j} x_{i,j,t} \cdot Y_t, S_t \right) \cdot P_t + \max \left(\sum_{i,j} x_{i,j,t} \cdot Y_t - S_t, 0 \right) \cdot \frac{P_t}{2} \right) - \sum_t \sum_{i,j} x_{i,j,t} \cdot C_t \quad (2)$$

5.1.4 约束条件的确定

约束条件一：种植面积的限制。

每个地块的种植面积不能超过其实际面积：

$$\sum_{t \in \text{作物}} x_{i,j,t} \leq A_i$$

其中, $x_{i,j,t}$ 表示第 i 个地块在第 j 季节种植第 t 种作物的面积 (单位: 亩)

约束条件二: 避免连续重茬种植。

为保持土壤肥力, 需要避免连续两年在同一土地种植相同作物:

$$a_{i,j,t} + a_{i,j+1,t} \leq 1, \quad \forall i, t, j$$

其中, $a_{i,j,t}$ 表示, 第 i 个地块在第 j 季节是否种植第 t 种作物 (1 为是, 0 为否), $a_{i,j+1,t}$ 第 i 个地块在第 $j+1$ 季节是否种植第 t 种作物 (1 为是, 0 为否)。

约束条件三: 豆类作物种植约束。

考虑到豆类作物对土壤的固氮效应, 每三年内必须至少种植一次豆类作物以保持土壤肥力:

$$\sum_{j'=j}^{j+2} a_{i,j', \text{豆类}} \geq 1, \quad \forall i, j$$

其中, $a_{i,j', \text{豆类}}$ 表示在第 j' 年第 i 块地是否种植豆类作物

约束条件四: 每季种植面积的约束。

从田间管理者的角度来看, 每个季节每个季节的种植面积不能太分散, 从而保证管理的便利性:

$$\sum_{i \in \text{邻近地块}} x_{i,t,j} \geq \alpha \times x_{\text{总},t,j}, \quad \forall t, j$$

其中, α 是一个比例因子, 此处取 0.5, 用来控制作物分布的集中程度, $x_{\text{总},t,j}$ 表示作物 t 在季 j 的总种植面积。

综上所述, 建立最优的农作物种植策略规划模型:

$$\begin{cases} \text{Max} \sum_t \min \left(\sum_{i,j} x_{i,j,t} \cdot Y_t, S_t \right) \cdot P_t - x_{i,j,t} \cdot C_t \\ \text{Max} \sum \left(\min \left(\sum_{i,j} x_{i,j,t} \cdot Y_t, S_t \right) \cdot P_t + \max \left(\sum_{i,j} x_{i,j,t} \cdot Y_t - S_t, 0 \right) \cdot \frac{P_t}{2} \right) - \sum_t \sum_{i,j} x_{i,j,t} \cdot C_t \end{cases}$$

$$st. \begin{cases} \sum_{t \in \text{作物}} x_{i,j,t} \leq A_i \\ a_{i,j,t} + a_{i,j+1,t} \leq 1, \quad \forall i, t, j \\ \sum_{j'=j}^{j+2} a_{i,j', \text{豆类}} \geq 1, \quad \forall i, j \\ \sum_{i \in \text{邻近地块}} x_{i,t,j} \geq \alpha \times x_{\text{总},t,j}, \quad \forall t, j \end{cases}$$

针对约束条件较多、数据结构较为复杂的线性规划模型, 可以采用模拟退火、遗传算法等智能优化算法进行求解, 也可以选用现有的 MIP 模型求解器进行求解。由于 MIP 模型不能像线性规划问题一样用简单的单纯形法进行求解, 且考虑到智能算法的运行时间较长、易陷入局部最优解等缺点, 本文选用 GLPK 求解器进行求解。

5.2 优化模型的求解

5.2.1 GLPK 求解器原理

GLPK 是一个完全免费的开源库, 适合解决中小规模的线性规划和混合整数规划问题。该求解器使用分支定界法, 引入松弛变量、分支和界, 通过在每个子问题中引入“割平面”, 进一步收缩可行解空间, 加快求解速度。该求解器的求解过程如下表所示:

表 1 GLPK 求解过程

GLPK 求解最优种植策略

Step1: 加载数据, 提取地块编号、作物名称、土地面积等信息

Step2: 创建目标为最大化农作物种植利润模型

Step3: 定义决策变量, 为每个每个地块、每种作物、每个年份、每个季次, 定义以下决策变量:

- X: 作物的种植面积。
- A: 是否种植该作物 (二元变量)。
- sold: 正常销售的亩产量。
- waste: 滞销浪费的亩产量。

Step4: 基于题设情景, 设计目标函数

- $\text{Max sold} \times \text{单价} - \text{种植成本}$
- $\text{Max sold} \times \text{单价} + \text{waste} \times 0.5 \times \text{单价} - \text{种植成本}$

Step5: 添加必要约束条件

Step6: 调用 GLPK 优化器求解模型, 寻找最大化利润的种植计划

- 单纯形法: 忽略整数约束, 遍历顶点, 求解最大粮食类作物利润问题
- 分支定界法: 根据二元变量的取值 (0 或 1) 构建分支树, 逐步收缩解空间, 进行上下界更新和剪枝, 加速求解最优整数解

5.2.2 求解结果

1. 迭代结果

针对不同的滞销系数在 Python 中调用 GLPK 求解器进行运算。在 50%滞销时得到的最优种植策略下最大利润如下表所示: 、

- 滞销浪费:

表 2 GLPK 求解滞销浪费情景结果

数据类型	第一类	第二类	第三类	总和
运行时间 (sec)	0.00	0.00	0.00	0.00
求解节点 (个)	1	1	1	1
最大利润 (元)	14453740	30978140	22769296	68201176

- 降价出售:

表 3 GLPK 求解降价出售情景结果

数据类型	第一类	第二类	第三类	总和
运行时间 (sec)	0.00	0.00	0.00	0.00
求解节点 (个)	1	1	1	1
最大利润 (元)	26230412.5	48112490	22769296	97112198.5

其中最大利润是 2024-2030 年的总最大利润, 其余情形详见附录文件。根据最优策略结果可以看出, 当滞销部分降价出售时可以获得更大的利润。

2. 可视化展示

选取滞销系数为 50% 的利润最大化情景, 选取 2024 年-2027 年的种植策略进行可视化。在该分布图中, 横向代表 41 种不同作物, 纵向代表不同的地块编号, 该分布图包含第一季和第二季的所有情况。针对不同滞销系数的 8 种情形详细数据参见支撑材料。

- 滞销浪费:

在滞销浪费情景下, 如果有总产量超出预期销售量的部分, 则超出部分直接浪费。

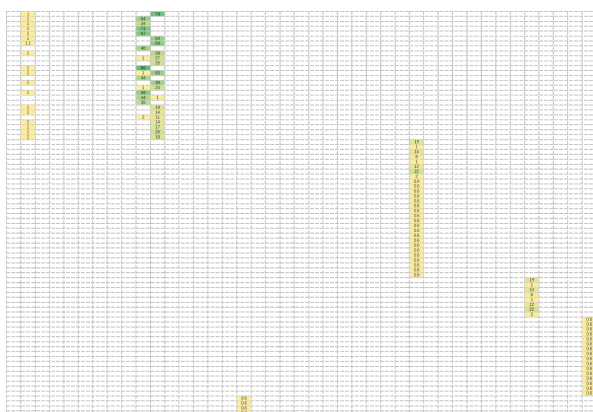


图 4 - 1 2024 年种植策略

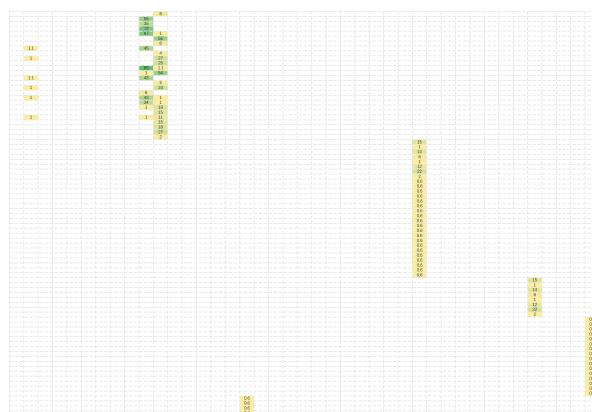


图 4 - 2 2026 年种植策略

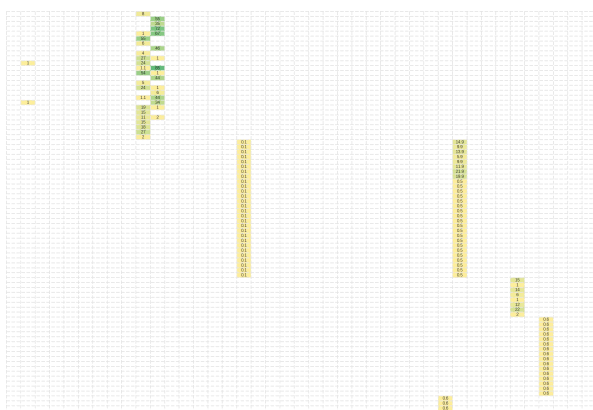


图 4 - 3 2025 年种植策略

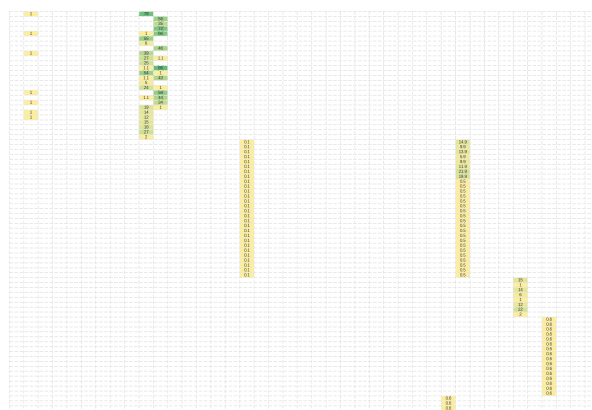


图 4 - 4 2027 年种植策略

图 5 滞销浪费情景下 2024-2027 年种植策略

通过纵向比对，可以快速查看连续两年间的作物种植情况。不同年份之间，主要差异表现为在不同年份中黑豆的种植差异较明显。而 2024 年和 2025 年相比，避免了重茬种植。此外，2024 年粮食作物与豆类作物共同种植，有利于维护土壤肥力，而 2025 年种植豆类作物减少，这也可能是出于利润最大化的同时，保证尽量种植豆类作物的目的。

同种地块上不同作物的组合种植为种植策略提供了丰富的多样性。

• 降价出售：

在降价出售情景下，如果有总产量超出预期销售量的部分，则超出部分按原价的 50% 销售。

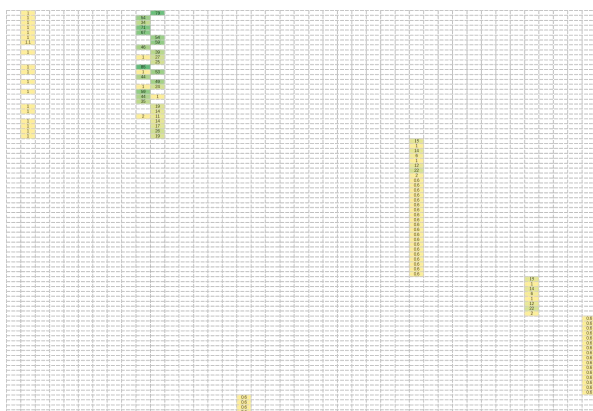


图 5 - 1 2024 年种植策略

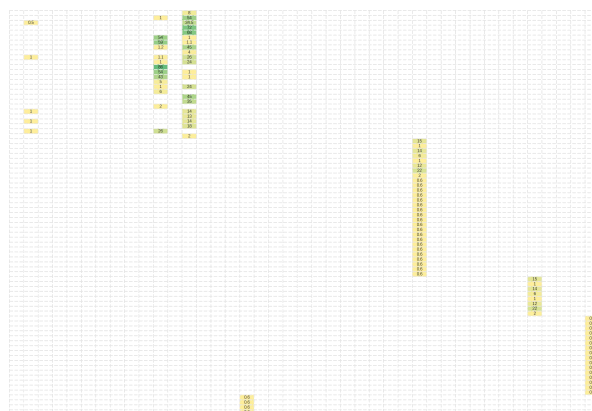


图 5 - 2 2026 年种植策略

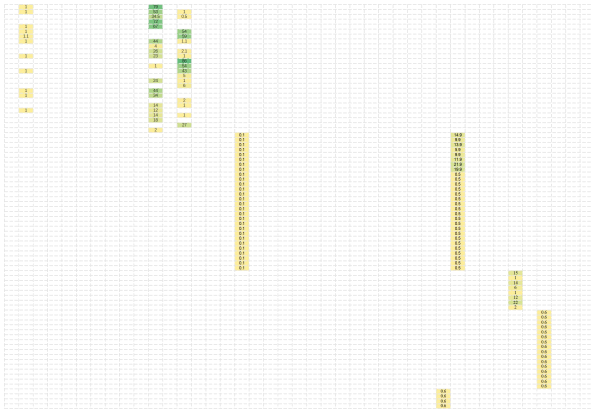


图 5 - 3 2025 年种植策略

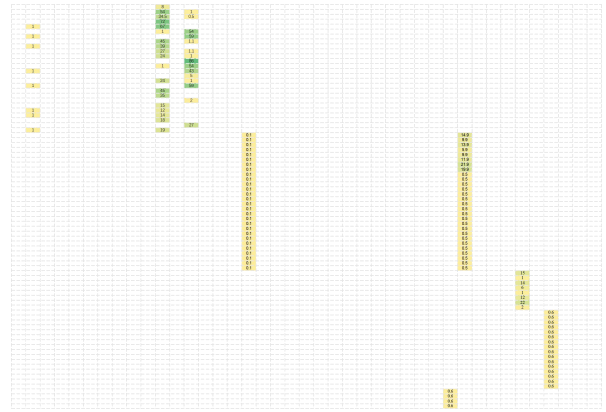


图 5 - 4 2027 年种植策略

图 6 降价销售情景下 2024-2027 年种植策略

过纵向比对，可以快速查看连续两年间的作物种植情况。不同年份之间，主要差异表现为在不同年份中黑豆的种植差异和豇豆的种植查阅比较明显。2026 年和 2027 年相比，避免了重茬种植。此外，大棚中都会选择种植食用菌类以获取更大利益。

出于田间管理的便利性，以上两种情形下种植的农作物类型较为集中。

六、 问题二模型的建立与求解

6.1 基于蒙特卡洛模拟与置信区间的波动情景设计

本题需要考虑农业发展过程中各种不确定性与潜在风险，因此首先需要设计各种变量不同的波动情景。蒙特卡洛模拟是一种基于随机抽样的计算方法，通过重复随机抽样来估计可能的结果分布和数值解。本题需要根据实际情况，考虑 2024-2030 年间种植业发展过程中的各种不确定性与潜在风险，本文选择蒙特卡洛模拟设计情景有以下优势：

- **风险分析：**通过模拟不同的风险因素和不确定性，可以评估种植策略在题目所示各种可能情况下的表现，从而更好地理解潜在的风险。

- **决策支持：**在某些农作物预期销售量、亩产量和销售价格不确定的环境中，蒙特卡洛模拟可以帮助决策者看到不同策略的潜在结果，为决策提供更多信息。

根据题意，整理汇总随时间变化的变量如表 4 所示。

表 4 第二问的指定变量波动情况

计算方法	变量名	变化情况
置信区间随机采样	小麦预期销售量	年均增长率 5%~10%
	玉米预期销售量	年均增长率 5%~10%
	食用菌销售价格	每年约下降 1%~5%
	农作物亩产量	每年约有±10%变化
	其他农作物预期销售量	相较 2023 年约有±5%变化
稳定变化	农作物种植成本	年均增长率为 5%
	粮食类作物销售价格	保持稳定
	蔬菜类作物销售价格	年均增长率为 5%
	羊肚菌的销售价格	年均下降 5%

6.1.1 稳定变化的变量计算

- 农作物种植成本：

$$C_{t,i} = C_{t,i-1} \times (1 + 5\%) \quad (3)$$

- 蔬菜类作物销售价格：

$$P_{\text{蔬菜类},i} = P_{\text{蔬菜类},i-1} \times (1 + 5\%) \quad (4)$$

- 羊肚菌销售价格：

$$P_{\text{羊肚菌},i} = P_{\text{羊肚菌},i-1} \times (1 - 5\%) \quad (5)$$

6.1.2 置信区间随机采样

本文采用置信区间来限制波动范围，更加精确地模拟出波动情况不确定的农作物未来销售量、产量、种植成本和销售价格的变化：

定义 d_- 和 d_+ 表示变量的下限和上限，基于题中所给数据的波动范围和历史数据计算得到置信区间，根据此范围生成随机数。提供置信区间的随机采样来模拟不同变量的波动。

根据变量的历史数据，可以根据以下公式计算置信区间：

$$d_{\pm} = \bar{x} \pm z \frac{\sigma}{\sqrt{n}} \quad (6)$$

其中， $\bar{x}$ 表示变量的平均值。 σ 表示标准差， n 表示样本数量， z 表示对应置信水平的 z 值。

在蒙特卡洛模拟过程中，为每个波动变量定义对应的置信区间，在每次模拟中根据置信区间生成随机浮动，变量是在置信区间内随机波动且符合正态分布，该变化趋势如下图所示。

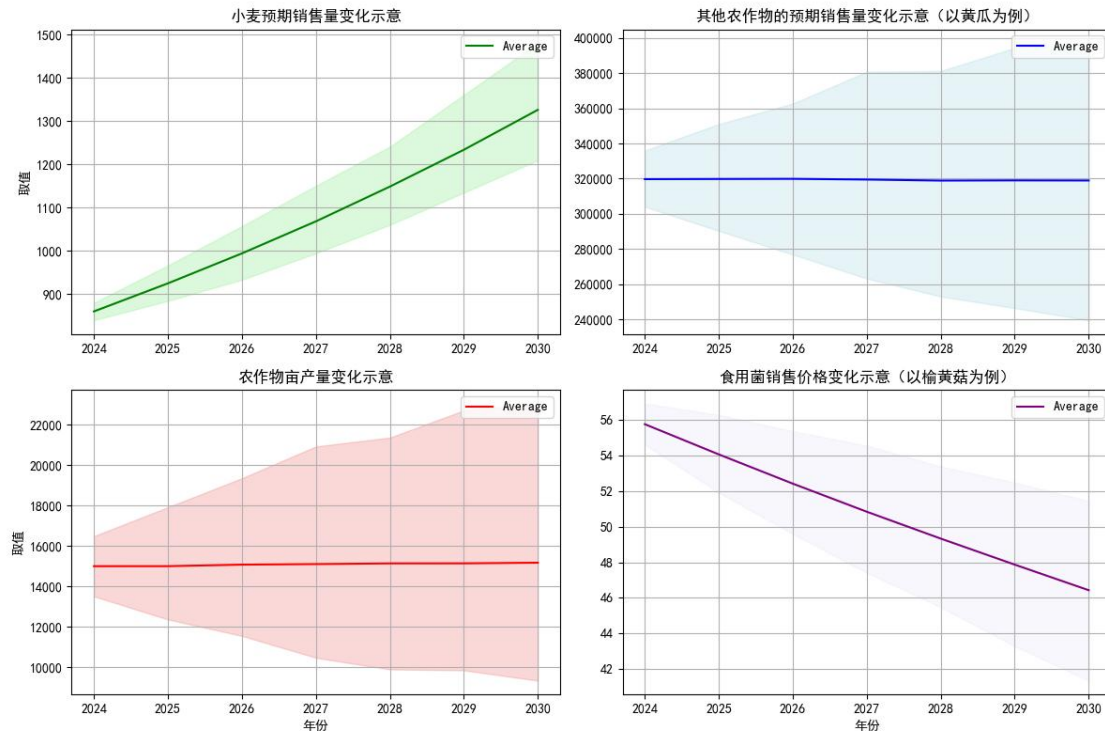


图 7 蒙特卡洛模拟过程中的波动范围

6.2 波动情形下的最优农作物种植策略规划模型

6.2.1 潜在风险评估

问题二要求在波动情景中，考虑潜在的种植风险。因此本文引入灾害因子改进种植策略规划模型。

灾害因子，可以用于量化模拟自然灾害对作物产量和种植成本的影响。通过对自然灾害程度的模拟，获得不同灾害情景下的收益，使得种植决策不仅仅考虑正常情况下的收益最大化，还需要面对自然灾害的潜在风险。在优化模型中，灾害因子可以辅助决策者得到更具现实意义的最优解。

现定义灾害因子 δ ，其范围在 $[0,1]$ 之间以反映不同程度的灾害。当 δ 取 0 时，表示没有灾害损失；当 δ 取 1 时，表示完全灾害损失，作物产量几乎为 0，甚至直接导致了巨大的经济损失； δ 越大表示灾害程度越严重。

在后续的规划模型中，灾害因子会影响作物的产量和种植成本，具体体现在：

1. 对产量的量化影响

通常，灾害因子越大，产量下降越多，其计算公式为：

$$Y_{\text{实际}} = Y \times (1 - \delta) \quad (7)$$

其中， $Y_{\text{实际}}$ 代表实际亩产量， Y 代表原亩产量。

2. 对种植成本的量化影响

灾害也会增加种植成本，灾害后往往需要额外的投入，如修复基础设施、抗灾物资等。因此，种植成本因灾害因子的增加而增长。其计算公式为：

$$C_{\text{实际}} = C \times (1 + \delta) \quad (8)$$

其中， $C_{\text{实际}}$ 代表实际种植成本， C 代表原种植成本。

3. 基于蒙特卡洛模拟生成灾害因子

在蒙特卡洛模拟中，灾害因子通常通过随机数生成，用来模拟未来几年内可能发生的灾害影响。这种随机模拟可以帮助我们：

- 量化灾害对产量和收益的影响；
- 评估不同灾害情境下的农作物收益波动；
- 调整目标函数，建立新的风险情景优化模型

6.2.2 决策变量的确定

记第 i 个地块在第 j 季节种植第 t 种作物的面积（单位：亩）为 $x_{i,j,t}$ ；记第 i 个地块在第 j 季节是否种植第 t 种作物（1 为是，0 为否）。

6.2.3 目标函数的确定

基于第一问的滞销浪费情景，综合引入的灾害因子，重新制定目标函数：

$$\text{Max} \sum_t \min \left(\sum_{i,j} x_{i,j,t} \cdot Y_t \cdot (1 - \delta), S_t \right) \cdot P_t - x_{i,j,t} \cdot C_t \cdot (1 + \delta) \quad (9)$$

约束条件与第一问相同，此处不再过多赘述。综上可以得到第二问的最优种植策略规划模型：

$$\begin{aligned}
& \text{Max} \sum_t \min \left(\sum_{i,j} x_{i,j,t} \cdot Y_t \cdot (1-\delta), S_t \right) \cdot P_t - x_{i,j,t} \cdot C_t \cdot (1+\delta) \\
& \text{st.} \begin{cases} \sum_{t \in \text{作物}} x_{i,j,t} \leq A_i \\ a_{i,j,t} + a_{i,j+1,t} \leq 1, \quad \forall i, t, j \\ \sum_{j'=j}^{j+2} a_{i,j', \text{豆类}} \geq 1, \quad \forall i, j \\ \sum_{i \in \text{邻近地块}} x_{i,t,j} \geq \alpha \times x_{\text{总},t,j}, \quad \forall t, j \end{cases} \quad (10)
\end{aligned}$$

6.3 规划模型求解

6.3.1 基于蒙特卡洛模拟的 GLPK 求解过程

在第一问的 GLPK 求解基础上，引入蒙特卡洛模拟，具体求解过程如下表所示。

表 5 基于蒙特卡洛模拟的 GLPK 求解过程

基于蒙特卡洛模拟的 GLPK 求解最优种植策略
Step1: 蒙特卡洛模拟数值 初始化模拟次数 N 从 1 到 N: - 根据定义好的概率分布，生成一组灾害因子（随机抽样） - 将生成的灾害因子保存到数组中 Step2: 加载数据，提取地块编号、作物名称、土地面积等信息 Step3: 创建目标为最大化农作物种植利润模型 Step4: 为每个每个地块、每种作物、每个年份、每个季次，定义以下决策变量： • X: 作物的种植面积。 • A: 是否种植该作物（二元变量）。 • sold: 正常销售的亩产量。 • waste: 滞销浪费的亩产量。 Step5: 基于题设情景，设计目标函数 • Max sold×单价-种植成本 • Max sold×单价+waste×0.5×单价-种植成本 Step6: 添加必要约束条件 Step7: 调用 GLPK 优化器求解模型，寻找最大化利润的种植计划 • 单纯形法：忽略整数约束，遍历顶点，求解最大粮食类作物利润问题 • 分支定界法：根据二元变量的取值（0 或 1）构建分支树，逐步收缩解空间，进行上下界更新和剪枝，加速求解最优整数解

6.3.2 求解结果

1. 最优种植策略

可视化最优种植策略结果图，我们可以得到在波动与风险情景下 2024-2027 年的种植策略。其中面积由大到小可表示为从绿色到红色的映射。从图中可以观察到，种植策略的主要差异表现为在 2024 与 2025 年相较于 2026 与 2027 的种植程度更为分散。

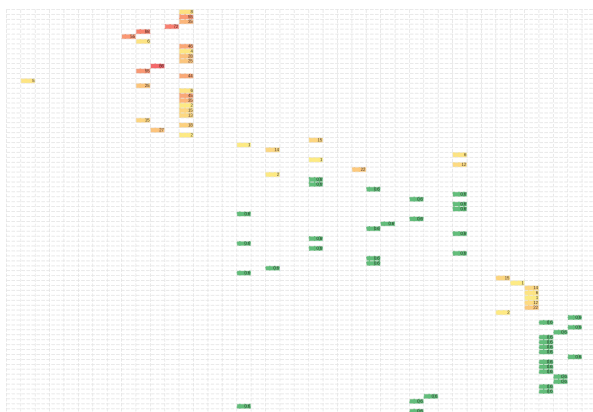


图 7- 1 2024 年种植策略

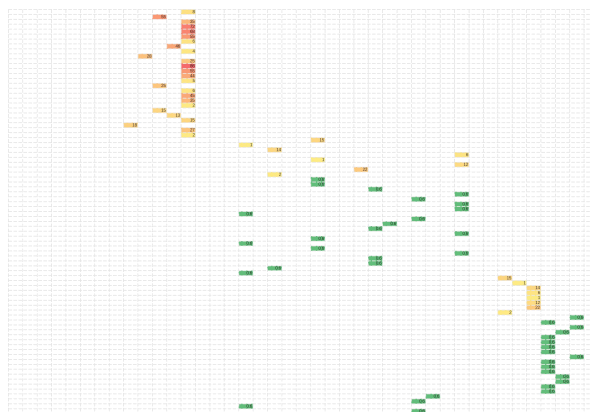


图 7- 2 2026 年种植策略

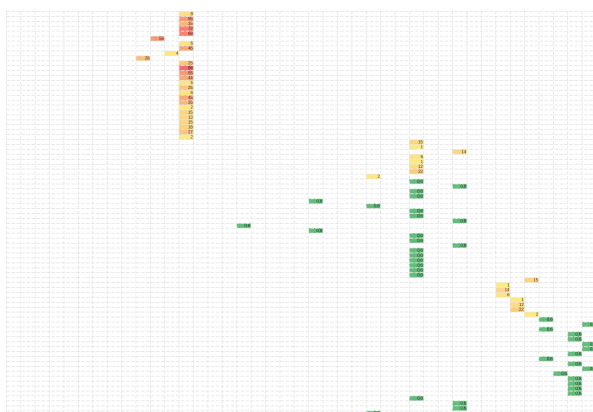


图 7- 3 2025 年种植策略

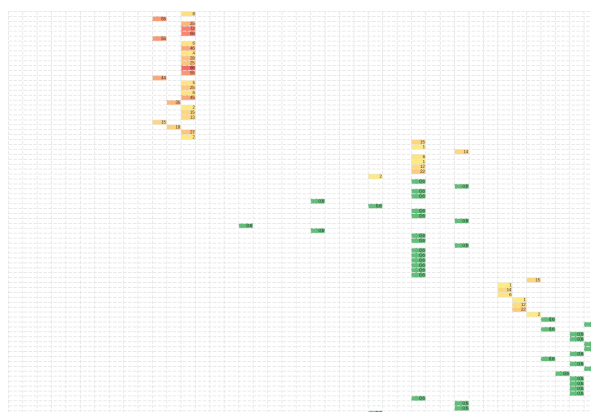


图 7- 4 2027 年种植策略

图 8 波动与风险情景下 2024-2027 年最优种植策略

2. 迭代结果

表 6 GLPK 求解波动情景种植优化模型结果

数据类型	2024	2025	2026	2027	2028	2029	2030
运行时间 (sec)	0.00	0.00	0.00	0.00	0.00	0.00	0.00
求解节点 (个)	1	1	1	1	1	1	1
最大利润 (元)	5.48e+06	5.07e+06	4.78e+06	5.08e+06	5.19e+06	5.14e+06	4.79e+06

七、 问题三模型的建立与求解

7.1 现实情景下农作物潜在关系分析

在现实种植业决策过程中，除了考虑市场潜在的风险与波动以外，还需要关注各种农作物之间的替代性与互补性，同时也要考虑农作物预期销量与销售价格、种植成本之间的相关性。在引入新的条件前，需要对替代性、互补性、相关性三个名词进行解释：

• **作物的替代性：**从种植业角度来看：当两种或多种农作物种植的预期收益相近、种植措施相似时，称它们存在替代性。从市场角度来看：当两种或多种农作物可以在市场上可以相互替代以满足消费者的某种需求，称它们存在替代性，即一种农作物价格上

涨，其替代作物，需求可能会增加。

• **作物的互补性：**从种植业角度来看：当两种或多种农作物一起种植时，有利于各自的生长，如可以减少种植病虫害、提供营养等，称它们存在互补性；从市场角度来看：当两种或多种农作物在市场上通常一起使用或消费，则称它们存在互补性。

• **预期销量与销售价格、种植成本的相关性：**当某种作物单价波动较大时，其预期销量也可能随之波动。同时种植成本也与销售价格有较强的关联关系。

在问题二的基础上，基于蒙特卡洛模拟，引入余弦相似度与交叉价格弹性，从农业和市场角度来探讨作物之间的替代互补关系。

7.1.1 基于余弦相似度的作物替代互补分析

余弦相似度是一种常用来衡量两个向量在多维空间中的方向相似性的方法，通过将个体的指标数据映射到向量空间，计算两个个体向量之间的内积空间夹角余弦值来度量它们之间的相似性。两个个体向量夹角余弦值越大，说明两个个体之间的相似度越高，两个个体夹角的余弦值越小，表明相似度越低。

在农业种植过程中，需要考虑适宜农作物种植的耕地类型、不同农作物之间的组合种植优势互补，同时也需要考虑农作物在市场上的表现。如果两个农作物的特征向量的余弦相似度很高，可能意味着它们具有相似的生长习性和市场反馈，因此可能具有替代性。相反，如果两个农作物的特征向量的余弦相似度很低，可能意味着它们在生长需求和市场反馈上有很大的不同，因此可能具有互补性。

通过余弦相似度可以帮助种植业决策者更好地理解不同农作物之间的关系，并据此做出更合理的种植策略和资源分配。

1. 确定特征维度

由于不同的耕地和不同季度可种植的农作物种类是固定的，在计算作物之间的余弦相似度时，首先根据不同的耕地类型和季度进行分类，同种耕地类型、同种季度种植的作物具有相似的生长特性。接着，以不同作物的**种植成本**、**亩产量**、**销售单价**作为输入特征，进行必要的标准化与归一化后构建特征向量。

2. 计算余弦相似度

使用向量 $\vec{V}_i$ 表示第*i*种作物的特征向量，对于两种不同的作物，二者的余弦相似度可以通过公式计算：

$$\cos_{i,j} = \frac{\vec{V}_i \cdot \vec{V}_j}{\|\vec{V}_i\| \times \|\vec{V}_j\|} \quad (11)$$

基于第二问的波动情景，计算粮食类（包括粮食类（豆类））、蔬菜类（包括蔬菜类（豆类））、食用菌类作物的余弦相似度。观察从农业种植角度，各个作物生长习性与市场反馈的相似程度。

3. 不同类型农作物余弦相似度分析

根据计算得出的农作物余弦相似度进行可视化：

对于粮食类作物，首先，粮食类（豆类）之间具有较强的余弦相似度，这说明不同的粮食豆类作物具有相似的市场反馈，在实际种植时可以互相替代。由于豆类作物有固氮效应，所以这种替代性可以更好地组织豆类在土地上的连续种植且不重茬。根据经验，小麦和玉米是华北地区粮食作物的主要来源，因此其市场反馈比较稳定，二者具有较高相似度、较强替代性。其次，荞麦、红薯、南瓜与其他作物均有较低的余弦相似度，特别是与豆类作物。说明这三种作物不可以轻易被其他作物替代，与豆类作物具有较强的互补性。

对于蔬菜类作物，首先，与粮食类作物类似，蔬菜豆类作物的替代性较强。其次，

空心菜、黄心菜、芹菜与豆类作物具有较强的互补性。从整体上来看，每种蔬菜均可以找出其替代作物，这与蔬菜品类繁多、性质类似以及需求波动较大有着一定联系。

对于食用菌作物，其各变量间相似度较低，具有较强的独立性。

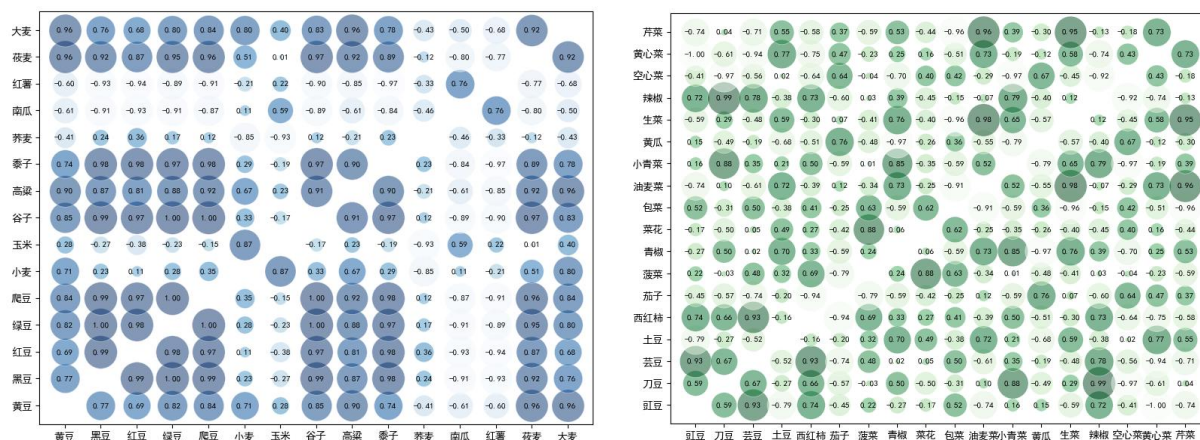


图 9 不同作物之间余弦相似度矩阵

7.1.2 基于交叉价格弹性的作物替代互补分析

1. 交叉价格弹性的计算

在经济学上，通常引入交叉弹性系数来探讨不同产品之间的替代性与互补性。交叉价格弹性计算公式为：

$$E_{ij} = \frac{\% \Delta S_i}{\% \Delta P_j} \quad (12)$$

其中， $E_{i,j}$ 表示产品 i 的需求对产品 j 价格变化的弹性， S_i 是农作物 i 的销售量， P_j 是农作物 j 的销售单价。

由于粮食类作物单价保持稳定，故不对其进行交叉价格弹性计算。基于第二问的波动情景和蒙特卡罗模拟方法，计算 2024-2030 年蔬菜类、食用菌类的交叉价格弹性系数，观察不同作物在市场上的替代与互补关系。

2. 替代性与互补性分析

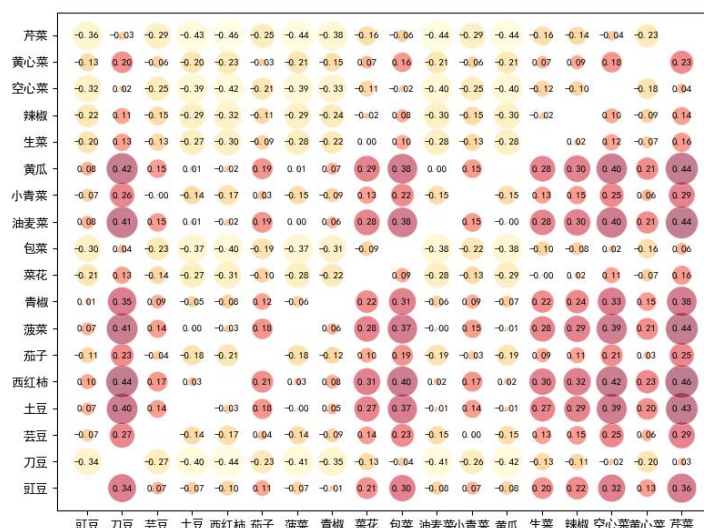


图 10 蒙特卡罗模拟的交叉价格弹性系数

- 交叉弹性为正，表示农作物 i 和 j 是替代品。即当产品 j 的价格上涨时，产品 i 的需求也增加。

• 交叉弹性为负，表示农作物 i 和 j 是互补品。即当产品 j 的价格上涨时，产品 i 的需求减少。

根据蒙特卡洛模拟的某情况计算作物间的交叉价格弹性系数。在图 10 的模拟情况中，豇豆与许多作物的交叉价格弹性系数均为负，此时豇豆与这些作物为互补品，即其他作物价格上涨时，豇豆的需求减少。

根据每年的实际情况，可以计算不同作物之间的交叉价格弹性。观察交叉价格弹性可以帮助决策者了解到市场上不同作物之间潜在的替代性和互补性，辅助建立种植业与市场关联关系，更好地制定决策。

7.1.3 基于 SEM 结构方程模型的相关性分析

结构方程模型（SEM）是根据潜变量形成因子，再通过自定义因子回归路径来进行分析。根据 SEM 模型可以分析变量之间的关系，同时可以处理复杂的因果关系和潜在变量。在实际生产过程中，预期销量、销售价格和种植成本往往存在一定的内部关联性，但该关系路径并不明晰，选用 SEM 模型可以很好地分析这一关系。

本文基于 SPSSPRO 平台，根据整合的数据进行 SEM 结构方程模型构建，求解结果如下：

1. 绘制结构方程模型路径图

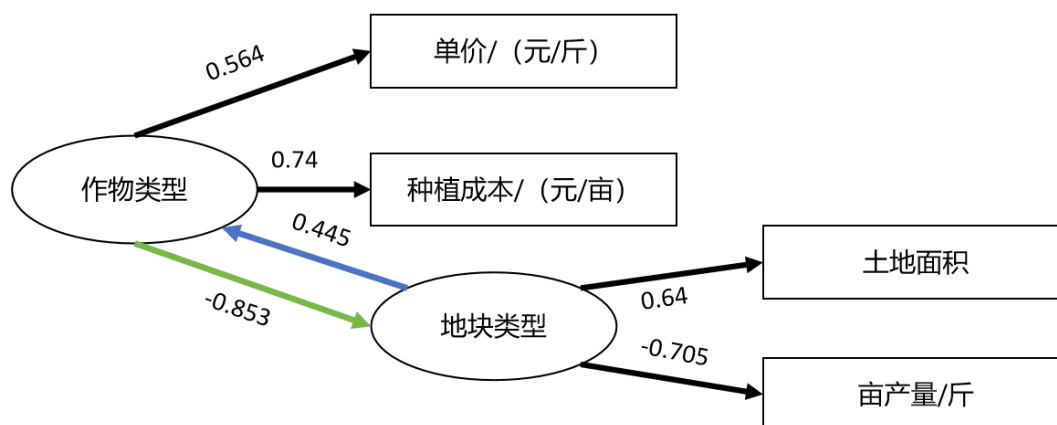


图 11 SEM 带权结构路径图

在该 SEM 结构方程模型中，变量之间的路径均已通过假设检验。其中，作物类型、地块类型是潜在变量（因子），单价、种植成本、土地面积、亩产量是观测变量。在各路径上是模型的标准化系数，用以分析变量之间的相关性。

2. 计算因子载荷系数

因子载荷矩阵用于表明潜在变量（因子）对观测变量（单价、种植成本、亩产量等）的影响程度。通过载荷系数，可以量化这些观测变量之间的关系，有利于进一步优化种植策略。

表 7 SEM 结构方程模型因子载荷系数表

因子	变量	非标准载荷系数	标准化载荷系数	z	S.E.	P
作物类型	单价/（元/斤）	1	0.564	-	-	-
	种植成本/(元/亩)	178.613	0.746	26.211	6.814	0.000***
地块类型	土地面积	1	0.647	-	-	-
	亩产量/斤	-152.431	-0.705	-29.023	5.252	0.000***

注：***、**、*分别代表 1%、5%、10%的显著性水平

根据因子载荷系数表，可以分析变量之间的相关性。

• 作物类型与单价和种植成本的关系：

单价与作物类型的非标准载荷系数为 1，表示其为参考值，不能直接用于计算，但标准化载荷系数为 0.564，表明二者相关性适中。

种植成本与作物类型之间的非标准载荷系数为 178.613，标准化载荷系数为 0.746，表明作物类型对种植成本有较强的正向影响。

- 地块类型与土地面积和亩产量的关系：

土地面积与地块类型的非标准载荷系数为 1，标准化载荷系数为 0.647，表明地块类型对土地面积的影响适中。

亩产量与地块类型的非标准载荷系数为-152.431，标准化载荷系数为-0.705，表明地块类型对亩产量有负向影响，这是因为某些类型的地块不适合高产量的作物，这与实际中某些作物只能种植在特定地块上的情形相符。

表 8 路径节点回归系数

Factor(潜变量)	→	分析项(显变量)	非标准化系数	标准化系数	标准误	Z	P
地块类型	→	作物类型	0.324	0.445	0.025	13.055	0.000***
作物类型	→	地块类型	-1.174	-0.853	0.055	-21.507	0.000***

注：***、**、*分别代表 1%、5%、10%的显著性水平

由模型路径系数表可知：

基于配对项地块类型->作物类型，显著性 P 值为 0.000***，在 1%的水平上显著，则拒绝原假设，因此此路径有效，其影响系数为 0.445。

基于配对项作物类型->地块类型，显著性 P 值为 0.000***，在 1%水平上显著，则拒绝原假设，因此此路径有效，其影响系数为-0.853。

3. 建立销售量与价格、成本的相关性

由于 SEM 的因子载荷反映的是因子对观测变量的影响，可以认为种植成本和单价对预期销售量有间接的影响，以此建立回归模型对预期销售量进行预测。

- 预期销售量与种植成本的关系：

根据因子载荷矩阵，种植成本与作物类型具有较强相关性，因此可以假设作物类型间接影响了销售量，而种植成本通过作物类型影响预期销售量。我们可以构建一个模型，将种植成本作为解释变量，来预测预期销售量。假设有如下线性回归模型：

$$S = \beta_0 + \beta_1 \times C$$

其中， S 表示预期销售量， C 表示种植成本，以下公式定义相同。 β_1 可以近似设为种植成本与作物类型的标准化载荷系数 0.746，通过影响作物类型间接影响销售量。

- 预期销售量与销售单价的关系：

由于单价与作物类型存在适中的正相关性。我们可以将单价作为另一个解释变量引入到模型中：

$$S = \beta_0 + \beta_1 \times C + \beta_2 \times P$$

其中， P 表示销售单价， β_2 可以设为 0.564，即种植成本与作物类型之间的标准化载荷系数。

- 预期销售量与地块特性的关系：

地块类型对亩产量有显著影响，根据标准化载荷系数，地块类型和亩产量之间的负相关系数为 -0.705，可以将其作为修正因子， Y 表示亩产量：

$$S = \beta_0 + \beta_1 \times C + \beta_2 \times P + \beta_3 \times Y$$

- 综合相关性模型：

基于上述分析，综合考虑不同变量对预期销售量的影响，我们可以构建如下的综合模型：

$$S = \beta_0 + 0.746 \times C + 0.564 \times P + (-0.705 \times \text{地块类型因子} \times Y) \quad (13)$$

7.2 基于聚类分析的最优作物选择模型

通过上述替代性、互补性与相关性分析，可以帮助我们进行作物筛选。为减少作物的种植类别，简化模拟计算过程，本文提出了基于土地系数与利润系数的作物评分模型，并以此为指标进行 K-means 聚类，筛选出 5 种最具代表性的作物，引入第二问的蒙特卡洛模拟规划模型中。最后，将模拟结果与第二问的结果进行对比。

7.2.1 基于土地系数与利润系数的作物评分模型

1. 土地系数计算

土地系数用于衡量作物种植的空间潜力，即每种作物之间的替代性或互补性，也包括为不同作物分配土地的合理性。该系数可以通过余弦相似度和交叉价格弹性得出，土地系数越高，表明作物对其他作物的替代性或互补性越强。其具体计算公式如下：

$$Land = 0.5 \times COS + 0.5 \times E \quad (14)$$

其中， $Land$ 表示土地系数， COS 表示作物的余弦相似度评分， E 表示交叉价格弹性评分。

2. 利润系数计算

利润系数用于衡量每种作物的潜在经济收益。通过 SEM 结构方程模型中的因子载荷和矩阵，可以得出种植成本、单价以及预期销售量之间的相关性，并以此建立利润系数计算公式：

$$Profit = 0.564 \times P + 0.746 \times C + 0.705 \times Y \quad (15)$$

其中， $Profits$ 表示利润系数， $[0.564, 0.746, 0.705]$ 系数矩阵是 SEM 结构方程中的因子载荷系数，反映了相关性的高低。

3. 作物评分计算

土地系数衡量作物的互补性或替代性，利润系数衡量其收益潜力。二者反映了作物种植时需要考虑的两个方面，设置相同的权重计算作物评分：

$$Grades = 0.5 \times Land + 0.5 \times Profits \quad (16)$$

该评分为后续的聚类分析奠定了基础。

7.2.2 基于 K-means 聚类的最佳作物选择

K-means 聚类是一种无监督学习算法，该算法将数据集划分为 K 个簇，每个簇由其质心代表。算法通过迭代过程最小化簇内点到质心的距离，直到质心位置不再发生显著变化。每个数据点被分配到最近的质心，直到所有质心和数据点的分配稳定为止。

本文将作物评分输入 K-means 模型中，对作物进行分类。首先，观察碎石图确定聚类数目：

从碎石图中可以看出，当 K 取值为 3 时，图形出现了较大的拐点，可以选择 3 作为聚类的簇数。接着将作物聚为 3 类，并找到每个类别中评分最高的作物作为聚类中心。根据聚类结果，选择评分最高的 5 种作物作为代表带入第二问的模型中进行蒙特卡洛模拟。

表 9 最优作物选择方案

作物名称	土地系数	利润系数
白灵菇	14892.187229	0.736932
黄瓜	12737.902406	0.733479
空心菜	11643.825369	0.719922
羊肚菌	8594.259000	0.682090

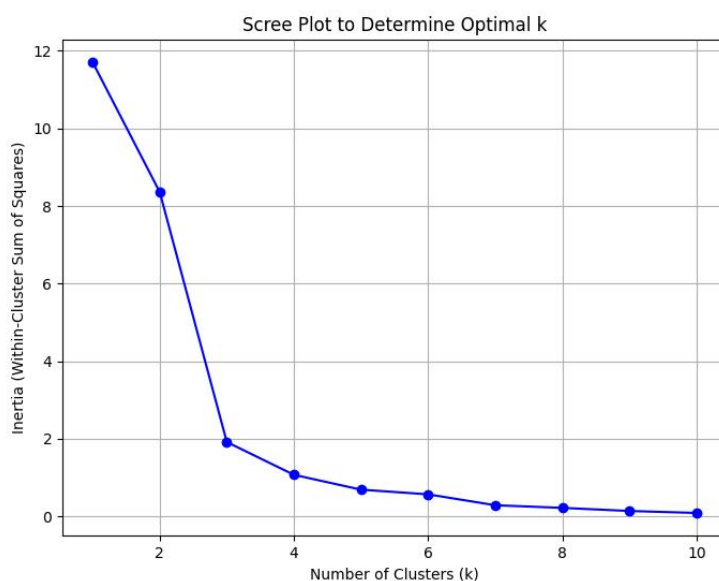


图 12 K-means 聚类碎石图

7.3 最优作物求解种植策略规划模型

根据最优作物种类，带入第二问的蒙特卡洛模拟规划模型中，求解利润最大化模型。由于最优作物都是蔬菜类或食用菌类，因此在选择第二问的蔬菜类最大利润与第三问模拟结果进行对比。二者均是在滞销系数为 0 时进行计算。

表 10 第二问与第三问结果对比

	求解时间 (sec)	最大总利润
第二问（蔬菜类）	0.00	3.15e+07
第三问（最优作物）	1.00	1.79e+09

观察该结果我们发现，第三问最优作物组合种植获得的最大总利润远高于第二问的情景，这可能有以下几个原因：

1. 在第三问中引入了作物之间的可替代性、互补性与销售量、单价、成本之间的相关性。同时进行了聚类分析与多种作物选择，这意味着作物选择不再是独立的，而是具有更为合理、更贴近现实的种植组合，因此有分散风险的能量，有利于提高总体收益的稳定性。

2. 第二问缺少对作物之间替代性互补性的讨论，只是单纯的种植优化，可能会导致其缺少对实际市场、经济与农业发展关系的综合考量。

第三问的求解过程更为灵活与贴近现实，从长期来看，因为考虑了经济因素的复杂性，可能具有更为稳健的结果。

八、鲁棒性检验

在计算各作物的余弦相似度时，我们采用的是 2023 年作物的相关数据，为了探究余弦相似度对数据集变化的敏感度，我们对作物数据集引入 5% 的噪声，并绘制余弦相似度变化程度矩阵进行观察。

[illegible]

图 13 加入噪声的余弦相似度变化矩阵

从图中的余弦相似度矩阵变化可以看出，最大的变化程度为 0.14。余弦相似度的范围在-1 到 1 之间，0.14 的变化在该范围内相对较小，表明数据对噪声的影响是相对稳健。这也意味着数据在当前特征维度是具有较高的分辨能力，轻微扰动不会大幅改变作物之间的相似关系。可以证明我们的模型具有较好的鲁棒性。

九、 模型的评价、改进与推广

9.1 模型的优点

1. 综合考虑了滞销情况以及灾害因素，模型系统全面，设置了不同的滞销率以及对灾害因子进行随机处理，分别分析不同情况对结果的影响，提高了模型的实用性和真实性。
2. 深入考虑现实生活的情况，站在农民和市场的角度利用交叉弹性系数和余弦相似度考虑农作物的可替代性和互补性，通过交叉弹性系数，可以更准确地分析市场对价格变化的反应，而余弦相似度可以帮助理解作物特性之间的相似性，两者结合可以提供更全面的市场分析，提高了模型对的准确性。
3. 本文在不同情况下使用规划模型，并利用 **GLPK** 求解器对模型进行求解，模型简洁性强，实用性更高，此外，本文利用蒙特卡洛进行仿真模拟，使得模型更加客观且真实，模型的结果的稳定性以及可靠性提高。

9.2 模型的缺点

1. 本文以 2023 年的销售范围的中位数作为该年该作物的销售单价，虽然极大程度避免了极端值的影响，但可能存在与该年该种作物销售单价的出入。
2. 在进行潜在风险的评定时，我们利用随机灾害因子的方法来进行测算，虽然提高了模型的适用性，但这在一定程度上可能与当年的实际情况不相符。
3. 在进行规划模型的时，对约束条件的研究可能不够深入。

9.3 模型的改进

1. 在对规划模型进行结果的求解时，可以考虑加入智能优化算法进行模型的求解，提高算法的效率。
2. 在对规划模型进行建立时，可以考虑增加更多的约束条件，使得模型更贴近现实情况。

9.4 模型的推广

该模型不仅可以用于农作物的种植策略，对于其他类似策略安排的问题均可以进行求解，譬如安排公共交通路线和时刻表，以最大化乘客满意度和运营效率为目标函数，如何在多个项目和任务之间分配时间和资源，以按时完成和最小化控制成本为目标函数等。

十、 参考文献

- [1] 乔立娟.蔬菜产业生产经营主体风险管理研究[D].河北农业大学,2014.
- [2] 许竹青,李岩.基于农户的能源作物种植成本收益及替代性研究——以云南省临沧市膏桐种植为例[J].林业经济问题, 2009.DOI:CNKI:SUN:LYJW.0.2009-02-008.
- [3] 阮清廉,刘喜,江玲,等.越南水稻生产概况及中越水稻生产互补性分析[J].杂交水稻, 2017, 32(6):11.DOI:10.16267/j.cnki.1005-3956.20170712.147.
- [4] 王侃,汪波.我国化肥的需求价格弹性分析[J].中国农机化, 2007(5):3.DOI:10.3969/j.issn.1006-7205.2007.05.009.
- [5] 刘云飞,周利,张春风,等.基于全谱数据库计算指纹图谱相似度新方法的探索研究[J].中医药学报, 2012.DOI:CNKI:SUN:ZYXB.0.2012-02-016.
- [6] 魏德云,陆军,谢银娟,金天澍.大棚蔬菜种植经营风险综合判断方法研究.中国农机化学报.第37卷第12期2016年12月
- [7] 刘亚钊 王秀清 日本生鲜蔬菜进口市场及其需求弹性分析 中国农业大学经济管理学院北京. 农业技术经济 2007年第2期
- [8] 陈军,陈祥云 生鲜蔬菜产地滞销预测模型与应用 重庆交通大学经济与管理学院
- [9] 江苏农业科学 2018年第46卷第2期
- [10] 郭梁, Andreas Wilkes, 于海英, 许建初中国主要农作物产量波动影响因素分析 植物分类与资源学报 2013, 35 (4): 513 ~ 521

附录

附录 1

介绍：支撑材料的文件列表

代码全部使用 `python` 语言

第二问代码

第三问代码

第一问代码

 abc单季.xlsx

 def1蔬菜.xlsx

 def2蔬菜.xlsx

 merged_data_with_years.xlsx

 monte_carlo_粮食类.xlsx

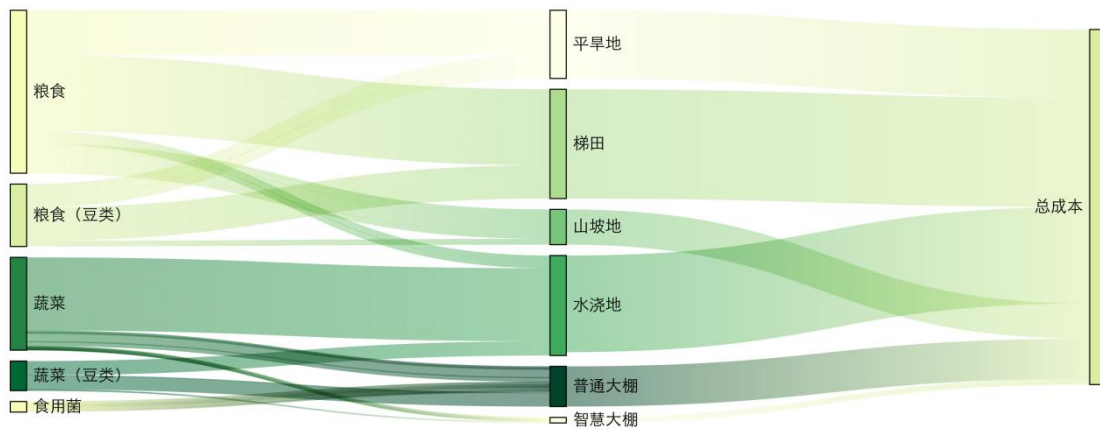
 monte_carlo_蔬菜类.xlsx

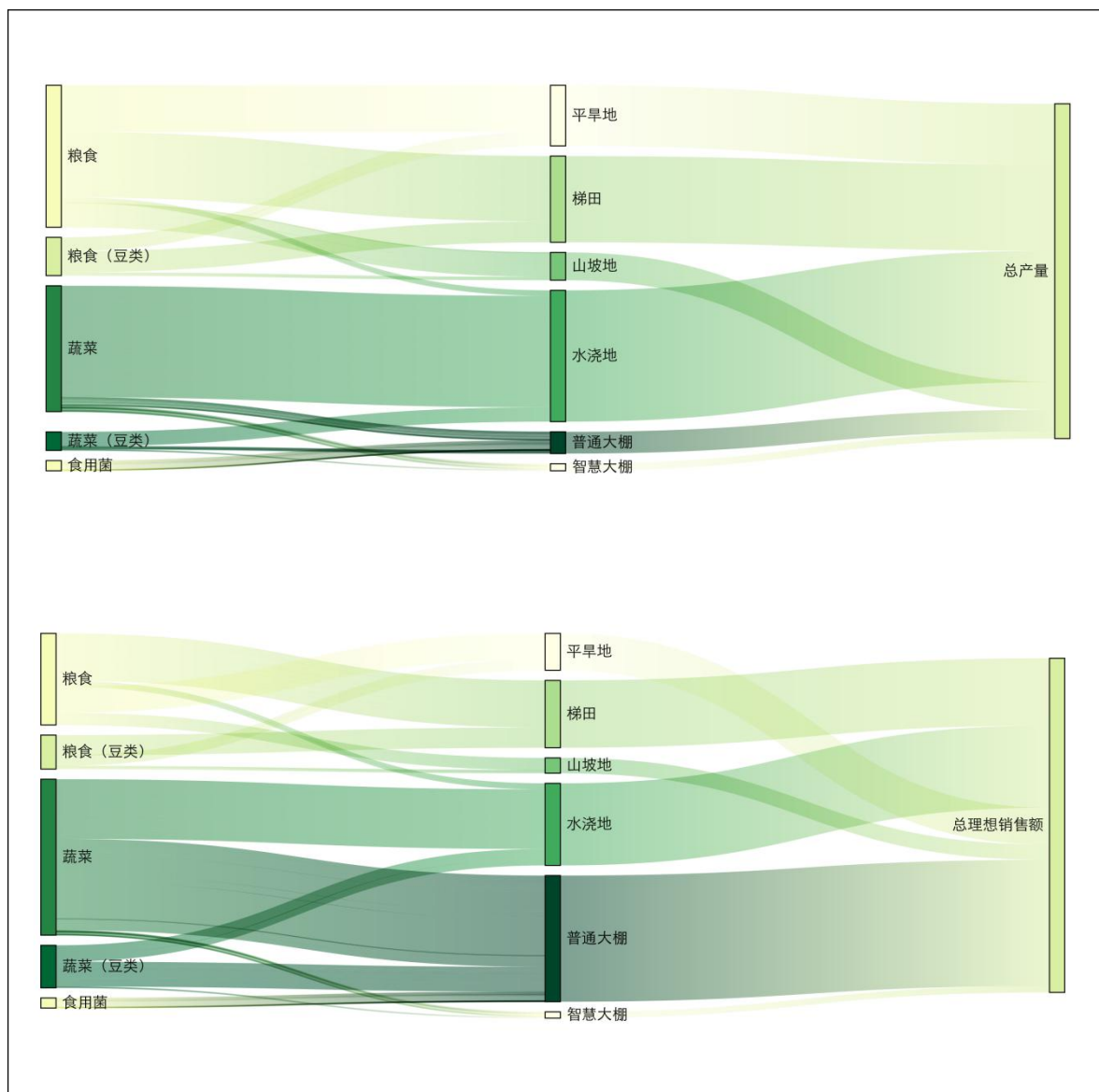
 数据预处理.xlsx

 数据整合2.xlsx

数据可视化桑基图

对总成本、总预期销售额同样绘制桑基图





第一问 (1) 代码 附录只给出粮食类作物的求解，蔬菜类的请看参考资料

针对 abc 地块粮食类单季作物的优化求解（浪费情况）

```
from pyscipopt import Model, quicksum
import pandas as pd
```

```
# 加载数据文件
```

```
abc_file_path = 'E:/国赛 2024/1 数据/abc 单季.xlsx'
```

```
abc_data = pd.read_excel(abc_file_path)
```

```
# 定义种植年限
```

```
years_hou = range(2023, 2031)
```

```
seasons = ['第一季'] # 季次
```

```
# 定义滞销系数 (例如: 0.25, 0.5, 0.75, 1.0)
```

```

wastage_factor = 0.25 # 滞销量系数

# =====
# 第一阶段：粮食类作物优化（ABC 地块），滞销量浪费
# =====
grain_model = Model("Maximize_Grain_Profit")

# 定义决策变量
grain_sold = {} # 正常销售的亩产量
grain_waste = {} # 滞销浪费的亩产量
X = {} # 种植面积决策变量
A = {} # A 变量表示是否种植作物，二元变量

for idx, row in abc_data.iterrows():
    land = row['土地编号']
    crop = row['作物名称']
    area = row['土地面积']

    for year in years_hou:
        for season in seasons:
            # 定义种植面积决策变量
            X[(land, crop, year, season)] =
grain_model.addVar(name=f'X_{land}_{crop}_{year}_{season}', lb=0, ub=area)
            # 定义是否种植的二元变量 A
            A[(land, crop, year, season)] = grain_model.addVar(vtype='B',
name=f'A_{land}_{crop}_{year}_{season}')
            # 定义正常销售的亩产量
            grain_sold[(land, crop, year, season)] =
grain_model.addVar(name=f'grain_sold_{land}_{crop}_{year}_{season}', lb=0)
            # 定义滞销浪费的亩产量
            grain_waste[(land, crop, year, season)] =
grain_model.addVar(name=f'grain_waste_{land}_{crop}_{year}_{season}', lb=0)

# 设置目标为最大化粮食类作物的利润
grain_objective = quicksum(
    grain_sold[(row['土地编号'], row['作物名称'], year, season)] * row['单价/（元/斤）
'] - # 销售部分按市场单价计算
    X[(row['土地编号'], row['作物名称'], year, season)] * row['种植成本/(元/亩)'] #
减去种植成本
    for idx, row in abc_data.iterrows()
    for year in years_hou
    for season in seasons
)

```

```

grain_model.setObjective(grain_objective, "maximize")

# =====
# 添加约束条件
# =====

# 约束条件一：种植面积不能超过地块面积
for idx, row in abc_data.iterrows():
    land = row['土地编号']
    area = row['土地面积']
    for year in years_hou:
        for season in seasons:
            grain_model.addCons(
                quicksum(X[(land, crop, year, season)] for crop in abc_data['作物名
称'].unique()) <= area
            )

# 约束条件二：避免连续两年种植相同作物
for land in abc_data['土地编号'].unique():
    for crop in abc_data['作物名称'].unique():
        for year in range(2023, 2030): # 循环到 2030 年，因为 2031 年没有后续年
            限
                for season in seasons:
                    grain_model.addCons(
                        X[(land, crop, year, season)] + X[(land, crop, year + 1, season)]
                        <= abc_data[abc_data['土地编号'] == land].iloc[0]['土地面积']
                    )

# 约束条件三：产量分配约束，确保销售量和滞销量的和与滞销系数一致
for idx, row in abc_data.iterrows():
    land = row['土地编号']
    crop = row['作物名称']
    for year in years_hou:
        for season in seasons:
            total_yield = X[(land, crop, year, season)] * row['亩产量/斤']
            # 销售量 = 总产量 * (1 - 滞销系数)
            grain_model.addCons(
                grain_sold[(land, crop, year, season)] == total_yield * (1 -
wastage_factor)
            )
            # 滞销量 = 总产量 * 滞销系数
            grain_model.addCons(
                grain_waste[(land, crop, year, season)] == total_yield *
wastage_factor
            )

```

```

    )

# 约束条件四：三年内至少种一次豆类作物
grain_bean_crops = ['黄豆', '黑豆', '红豆', '绿豆', '爬豆'] # 假设豆类作物
for land in abc_data['土地编号'].unique():
    for start_year in range(2024, 2030, 3): # 每三年一个周期检查
        grain_model.addCons(
            quicksum(
                X[(land, crop, year, season)]
                for crop in grain_bean_crops
                for year in range(start_year, start_year + 3)
                for season in seasons
            ) >= 1
        )

# 约束条件五：每个季节的种植面积不小于地块面积的 0.5 倍
for land in abc_data['土地编号'].unique():
    for year in years_hou:
        for season in seasons:
            area = abc_data[abc_data['土地编号'] == land].iloc[0]['土地面积']
            grain_model.addCons(
                quicksum(X[(land, crop, year, season)] for crop in abc_data['作物名
称'].unique()) >= 0.5 * area
            )

# =====
# 求解模型
# =====
grain_model.optimize()

# =====
# 保存粮食类作物求解结果
# =====
if grain_model.getStatus() == "optimal":
    print("Optimal grain solution found")
    grain_result = []
    for idx, row in abc_data.iterrows():
        land = row['土地编号']
        crop = row['作物名称']
        for year in years_hou:
            for season in seasons:
                var_value = grain_model.getVal(X[(land, crop, year, season)])
                sold_value = grain_model.getVal(grain_sold[(land, crop, year,
season)])

```



```

        waste_value = grain_model.getVal(grain_waste[(land, crop, year,
season)])

        if var_value > 0: # 仅保存种植面积大于 0 的情况
            grain_result.append({
                '土地编号': land,
                '作物名称': crop,
                '年份': year,
                '季节': season,
                '种植面积': var_value,
                '销售产量': sold_value,
                '滞销产量': waste_value
            })

    df_grain_result = pd.DataFrame(grain_result)
    df_grain_result.to_excel("grain_optimized_result_wastage0.25.xlsx", index=False)
else:
    print("No optimal solution found for grain crops.")

```

第一问（2）代码

针对 abc 地块粮食类单季作物的优化求解（半价出售）

```

from pycscpt import Model, quicksum
import pandas as pd

# 加载数据文件
abc_file_path = 'E:/国赛 2024/1 数据/abc 单季.xlsx'
abc_data = pd.read_excel(abc_file_path)

# 定义种植年限
year_hou = range(2023, 2031)
seasons = ['第一季'] # 季次

# 定义滞销系数 (例如: 0.25, 0.5, 0.75, 1.0)
wastage_factor = 0.25 # 滞销量系数

# =====
# 第一阶段: 粮食类作物优化 (ABC 地块), 滞销量浪费
# =====
grain_model = Model("Maximize_Grain_Profit")

# 定义决策变量
grain_sold = {} # 正常销售的亩产量
grain_waste = {} # 滞销浪费的亩产量
X = {} # 种植面积决策变量

```

```

A = {} # A 变量表示是否种植作物，二元变量

for idx, row in abc_data.iterrows():
    land = row['土地编号']
    crop = row['作物名称']
    area = row['土地面积']

    for year in year_hou:
        for season in seasons:
            # 定义种植面积决策变量
            X[(land, crop, year, season)] =
grain_model.addVar(name=f'X_{land}_{crop}_{year}_{season}', lb=0, ub=area)
            # 定义是否种植的二元变量 A
            A[(land, crop, year, season)] = grain_model.addVar(vtype='B',
name=f'A_{land}_{crop}_{year}_{season}')
            # 定义正常销售的亩产量
            grain_sold[(land, crop, year, season)] =
grain_model.addVar(name=f'grain_sold_{land}_{crop}_{year}_{season}', lb=0)
            # 定义滞销浪费的亩产量
            grain_waste[(land, crop, year, season)] =
grain_model.addVar(name=f'grain_waste_{land}_{crop}_{year}_{season}', lb=0)

# 设置目标为最大化粮食类作物的利润

grain_objective = quicksum(
    (grain_sold[(row['土地编号'], row['作物名称'], year, season)] * row['单价/(元/斤)
'] + # 销售部分收入
    grain_waste[(row['土地编号'], row['作物名称'], year, season)] * wastage_factor *
row['单价/(元/斤)']) - # 滞销部分收入
    X[(row['土地编号'], row['作物名称'], year, season)] * row['种植成本/(元/亩)']) #
种植成本
    for idx, row in abc_data.iterrows()
    for year in year_hou
    for season in seasons
    if (row['土地编号'], row['作物名称'], year, season) in X
)
grain_model.setObjective(grain_objective, "maximize")

# =====
# 添加约束条件
# =====

# 约束条件一：种植面积不能超过地块面积
for idx, row in abc_data.iterrows():

```

```

land = row['土地编号']
area = row['土地面积']
for year in year_hou:
    for season in seasons:
        grain_model.addCons(
            quicksum(X[(land, crop, year, season)] for crop in abc_data['作物名
称'].unique()) <= area
        )

# 约束条件二：避免连续两年种植相同作物
for land in abc_data['土地编号'].unique():
    for crop in abc_data['作物名称'].unique():
        for year in range(2023, 2030): # 循环到 2030 年，因为 2031 年没有后续年
            限
                for season in seasons:
                    grain_model.addCons(
                        X[(land, crop, year, season)] + X[(land, crop, year + 1, season)]
                        <= abc_data[abc_data['土地编号'] == land].iloc[0]['土地面积']
                    )

# 约束条件三：产量分配约束，确保销售量和滞销量的和与滞销系数一致
for idx, row in abc_data.iterrows():
    land = row['土地编号']
    crop = row['作物名称']
    for year in year_hou:
        for season in seasons:
            total_yield = X[(land, crop, year, season)] * row['亩产量/斤']
            # 销售量 = 总产量 * (1 - 滞销系数)
            grain_model.addCons(
                grain_sold[(land, crop, year, season)] == total_yield * (1 -
wastage_factor)
            )
            # 滞销量 = 总产量 * 滞销系数
            grain_model.addCons(
                grain_waste[(land, crop, year, season)] == total_yield *
wastage_factor
            )

# 约束条件四：三年内至少种一次豆类作物
grain_bean_crops = ['黄豆', '黑豆', '红豆', '绿豆', '爬豆'] # 假设豆类作物
for land in abc_data['土地编号'].unique():
    for start_year in range(2024, 2030, 3): # 每三年一个周期检查
        grain_model.addCons(
            quicksum(

```

```

        X[(land, crop, year, season)]
        for crop in grain_bean_crops
        for year in range(start_year, start_year + 3)
        for season in seasons
    ) >= 1
)

# 约束条件五：每个季节的种植面积不小于地块面积的 0.5 倍
for land in abc_data['土地编号'].unique():
    for year in year_hou:
        for season in seasons:
            area = abc_data[abc_data['土地编号'] == land].iloc[0]['土地面积']
            grain_model.addCons(
                quicksum(X[(land, crop, year, season)] for crop in abc_data['作物名
称'].unique()) >= 0.5 * area
            )

# =====
# 求解模型
# =====
grain_model.optimize()

# =====
# 保存粮食类作物求解结果
# =====
if grain_model.getStatus() == "optimal":
    print("Optimal grain solution found")
    grain_result = []
    for idx, row in abc_data.iterrows():
        land = row['土地编号']
        crop = row['作物名称']
        for year in year_hou:
            for season in seasons:
                var_value = grain_model.getVal(X[(land, crop, year, season)])
                sold_value = grain_model.getVal(grain_sold[(land, crop, year,
season)])
                waste_value = grain_model.getVal(grain_waste[(land, crop, year,
season)])

                if var_value > 0: # 仅保存种植面积大于 0 的情况
                    grain_result.append({
                        '土地编号': land,
                        '作物名称': crop,
                        '年份': year,
                        '季节': season,

```

```

        '种植面积': var_value,
        '销售产量': sold_value,
        '滞销产量': waste_value
    })

    df_grain_result = pd.DataFrame(grain_result)
    df_grain_result.to_excel("grain_optimized_result_sale0.25.xlsx", index=False)
else:
    print("No optimal solution found for grain crops.")

```

第二问代码 附录只给出粮食类的代码，蔬菜类请看参考资料

Abc 粮食类一季作物蒙塔卡洛随机数值模拟

```

# Load the newly uploaded file for data processing
file_path_new = 'E:/国赛 2024/1 数据/abc 单季.xlsx'
data_2023_new = pd.read_excel(file_path_new)

# Define the number of simulations and years for the Monte Carlo simulation
num_simulations = 1000 # Number of simulations
num_years = 7 # Number of years for future data

# Set seed for reproducibility
np.random.seed(42)

# Define simulation conditions
wheat_corn_growth_rates = np.random.uniform(0.05, 0.10, (num_simulations,
num_years)) # 小麦和玉米
other_crops_sales_changes = np.random.uniform(-0.05, 0.05,
                                                (num_simulations, num_years))

# 其他作物
yield_changes = np.random.uniform(-0.10, 0.10, (num_simulations, num_years)) #
Yield changes
mushroom_price_change_rates = {
    '榆黄菇': np.random.uniform(-0.05, -0.01, (num_simulations, num_years)),
    '香菇': np.random.uniform(-0.05, -0.01, (num_simulations, num_years)),
    '白灵菇': np.random.uniform(-0.05, -0.01, (num_simulations, num_years)),
    '羊肚菌': -0.05 # Fixed price change rate for 羊肚菌
}

crop_growth_cost_rate = 0.05 # 农作物变化率(5%)
vegetable_price_growth_rate = 0.05 # 蔬菜价格 (5%)

# Extract crop names from the dataset
crops = data_2023_new.columns.tolist() # Assuming the crops are the column headers
in the dataset

```

```

# Create a list of crops for yield, cost, and sales changes
yield_crops = crops
sales_crops = crops
cost_growth_crops = crops

# Create a DataFrame for the simulation results
simulation_data = []

for i in range(num_simulations):
    for year in range(num_years):
        simulation_entry = {
            'Simulation': i + 1,
            'Year': 2023 + year,
            'Wheat_Corn_Growth_Rate': wheat_corn_growth_rates[i, year],
            'Other_Crops_Sales_Change': other_crops_sales_changes[i, year],
            'Yield_Change': yield_changes[i, year],
            'Corn_Growth_Cost_Rate': crop_growth_cost_rate,
            'Vegetable_Price_Growth': vegetable_price_growth_rate,
            '羊肚菌_Price_Change': mushroom_price_change_rates['羊肚菌']
        }
        # Add mushroom price changes to the simulation entry
        for mushroom in ['榆黄菇', '香菇', '白灵菇']:
            simulation_entry[f'{mushroom}_Price_Change'] =
mushroom_price_change_rates[mushroom][i, year]

        simulation_data.append(simulation_entry)

df_simulation = pd.DataFrame(simulation_data)

# Save the updated simulation data to an Excel file
monte_carlo_simulation_path = 'E:/国赛 2024/1 数据/monte_carlo_1.xlsx'
df_simulation.to_excel(monte_carlo_simulation_path, index=False)

monte_carlo_simulation_path

```

第二问代码

Abc 粮食类单季+蒙特卡洛随机数+灾害因子的种植优化求解

```

import numpy as np
from pycipopt import Model, quicksum
import pandas as pd

```

```

# Load Monte Carlo simulation data

```

```

monte_carlo_file = 'E:/国赛 2024/1 数据/monte_carlo_simulation_2024_2030_final.xlsx'
monte_carlo_data = pd.read_excel(monte_carlo_file, sheet_name=None)

# Define the years and seasons_hou for the model
years = range(2024, 2031) # Years 2024-2030
seasons_hou = ['第一季'] # Single season

# Define wastage factor (滞销系数)
discount_factor = 0.5 # 0.75 corresponds to 50% wastage

# Bean crops
bean_crops = ['黄豆', '黑豆', '红豆', '绿豆', '爬豆']

# Function to generate disaster factor (灾害因子)
def generate_disaster_factor(num_samples):
    # 生成一个 0 到 1 之间的灾害因子，值越大代表灾害越严重
    return np.random.uniform(0, 1, num_samples)

# Function to optimize the veggie planting plan for a specific year with disaster factor and
wastage factor
def optimize_veggie_for_year_with_disaster(year_data, year):
    veggie_model = Model(f'Maximize_Veggie_Profit_{year}')

    # 生成灾害因子
    disaster_factors = generate_disaster_factor(len(year_data))

    # Define decision variables
    X = {} # Planting area for crop i on land j in season t
    yield_sold = {} # Sold yield
    yield_waste = {} # Wasted yield
    A = {} # Binary variable for whether crop is planted

    for idx, row in year_data.iterrows():
        land = row['土地编号']
        crop = row['作物名称']
        area = row['土地面积']
        disaster_factor = disaster_factors[idx] # 当前作物的灾害因子

        for season in seasons_hou:
            # Continuous variable for planting area
            X[(land, crop, year, season)] = veggie_model.addVar(lb=0, ub=area,
vtype='C', name=f'X_{land}_{crop}_{year}_{season}')
            # Binary variable for planting decision
            A[(land, crop, year, season)] = veggie_model.addVar(vtype='B',

```



```

name=f'A_{land}_{crop}_{year}_{season}')
    # Continuous variables for sold and wasted yield
    yield_sold[(land, crop, year, season)] = veggie_model.addVar(lb=0,
vtype='C', name=f'yield_sold_{land}_{crop}_{year}_{season}')
    yield_waste[(land, crop, year, season)] = veggie_model.addVar(lb=0,
vtype='C', name=f'yield_waste_{land}_{crop}_{year}_{season}')

    # Define objective: Maximize profit, adjusting for disaster factors and wastage factor
    veggie_objective = quicksum(
        (yield_sold[(row['土地编号'], row['作物名称'], year, season)] * (1 +
disaster_factors[idx]) * row['单价/(元/斤)'] - # 价格因灾害上涨
        (X[(row['土地编号'], row['作物名称'], year, season)] * row['种植成本/(元/
亩)'] * (1 - disaster_factors[idx]))) # 产量因灾害下降
        for idx, row in year_data.iterrows()
        for season in seasons_hou
        if (row['土地编号'], row['作物名称'], year, season) in X
    )

    veggie_model.setObjective(veggie_objective, "maximize")

# Constraints

# Constraint 1: Planting area and binary decision relationship
for land in year_data['土地编号'].unique():
    for crop in year_data['作物名称'].unique():
        for season in seasons_hou:
            if (land, crop, year, season) in X:
                veggie_model.addCons(X[(land, crop, year, season)] >= 0.1 *
A[(land, crop, year, season)])
                veggie_model.addCons(X[(land, crop, year, season)] <=
year_data[year_data['土地编号'] == land].iloc[0]['土地面积'] * A[(land, crop, year,
season)])

# 约束条件二：避免连续两年在同一地块种植相同作物
for land in year_data['土地编号'].unique():
    for crop in year_data['作物名称'].unique():
        for season in seasons_hou:
            if year > 2024: # 只有在年份大于 2024 时才应用这个约束
                if (land, crop, year - 1, season) in A:
                    veggie_model.addCons(A[(land, crop, year - 1, season)] +
A[(land, crop, year, season)] <= 1)

# Constraint 3: Ensure total planting area does not exceed available land
for land in year_data['土地编号'].unique():

```

```

        total_area = year_data[year_data['土地编号'] == land].iloc[0]['土地面积']
        for season in seasons_hou:
            veggie_model.addCons(
                quicksum(X.get((land, crop, year, season), 0) for crop in year_data['
作物名称'].unique()) <= total_area
            )

# Constraint 4: Yield allocation, ensuring sold + wasted yield equals total yield
for idx, row in year_data.iterrows():
    land = row['土地编号']
    crop = row['作物名称']
    for season in seasons_hou:
        veggie_model.addCons(
            yield_sold[(land, crop, year, season)] == X[(land, crop, year, season)]
* row['亩产量/斤'] * (1 - discount_factor)
        )
        veggie_model.addCons(
            yield_waste[(land, crop, year, season)] == X[(land, crop, year,
season)] * row['亩产量/斤'] * discount_factor
        )

# Solve the model
veggie_model.optimize()

# Save results if optimal solution is found
if veggie_model.getStatus() == "optimal":
    print(f'Optimal solution found for {year}')
    veggie_result = []
    for idx, row in year_data.iterrows():
        land = row['土地编号']
        crop = row['作物名称']
        for season in seasons_hou:
            if (land, crop, year, season) in X:
                var_value = veggie_model.getVal(X[(land, crop, year, season)])
                sold_value = veggie_model.getVal(yield_sold[(land, crop, year,
season)])
                waste_value = veggie_model.getVal(yield_waste[(land, crop,
year, season)])
                if var_value > 0: # Only save results with non-zero planting
area
                    veggie_result.append({
                        '土地编号': land,
                        '作物名称': crop,
                        '年份': year,

```

```

        '季次': season,
        '种植面积': var_value,
        '销售产量': sold_value,
        '滞销产量': waste_value
    })

    df_veggie_result = pd.DataFrame(veggie_result)
    return df_veggie_result
else:
    print(f"No optimal solution found for {year}")
    return None

# Optimize for each year based on the simulation data and save results
all_years_results = []
for year in range(2024, 2031):
    year_data = monte_carlo_data[f'Year_{year}']
    year_result = optimize_veggie_for_year_with_disaster(year_data, year)
    if year_result is not None:
        all_years_results.append(year_result)

# Combine all results into one dataframe and save as Excel
final_results_df = pd.concat(all_years_results, ignore_index=True)
final_results_df.to_excel("veggie_optimized_results_with_disaster_2024_2030_0.5.xlsx",
index=False)

```

第三问

余弦相似度计算

```

import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# 读取文件路径
file_path = 'E:/2024 国赛/merged_data_with_years.xlsx'
data = pd.read_excel(file_path)

# 筛选相关列
relevant_columns = ['作物名称', '单价/(元/斤)', '种植成本/(元/亩)', '亩产量/斤', '土地面积']
data_filtered = data[relevant_columns]

# 根据作物名称分组并计算均值
data_grouped = data_filtered.groupby('作物名称').mean()

# 标准化数据
data_standardized = (data_grouped - data_grouped.mean()) / data_grouped.std()

```

```
# 计算余弦相似度矩阵
similarity_matrix = pd.DataFrame(cosine_similarity(data_standardized),
                                index=data_grouped.index,
                                columns=data_grouped.index)

# 保存结果到文件
similarity_matrix.to_excel('E:/2024 国赛/作物余弦相似度矩阵.xlsx')
```

第三问

交叉弹性分析

```
import pandas as pd
import numpy as np

# 读取数据文件
file_path = 'E:/2024 国赛/merged_data_with_years.xlsx'
data = pd.read_excel(file_path)

# 筛选出相关列：作物名称、单价、亩产量、年份
relevant_columns = ['作物名称', '单价/（元/斤）', '亩产量/斤', '年份']
data_filtered = data[relevant_columns].copy() # 使用 .copy() 防止 SettingWithCopyWarning

# 计算每种作物在不同年份的需求变化（使用亩产量表示需求量）
data_filtered['需求变化'] = data_filtered.groupby('作物名称')['亩产量/斤'].pct_change()

# 计算每种作物在不同年份的价格变化
data_filtered['价格变化'] = data_filtered.groupby('作物名称')['单价/（元/斤）'].pct_change()

# 删除需求和价格变化为 NaN 的行（例如，第一年）
data_filtered.dropna(subset=['需求变化', '价格变化'], inplace=True)

# 初始化交叉价格弹性矩阵
crop_names = data_filtered['作物名称'].unique()
cross_price_elasticity = pd.DataFrame(index=crop_names, columns=crop_names)

# 计算交叉价格弹性
for crop_i in crop_names:
    for crop_j in crop_names:
        # 获取作物 i 的需求变化和作物 j 的价格变化
        demand_change_i = data_filtered.loc[data_filtered['作物名称'] == crop_i, '需
```

```

求变化'].values
    price_change_j = data_filtered.loc[data_filtered['作物名称'] == crop_j, '价格变化'].values

    # 确保需求变化和价格变化的长度一致
    min_length = min(len(demand_change_i), len(price_change_j))
    if min_length > 0:
        demand_change_i = demand_change_i[:min_length]
        price_change_j = price_change_j[:min_length]

        # 排除价格变化为 0 的情况，避免除以零错误
        non_zero_indices = price_change_j != 0
        if np.any(non_zero_indices):
            elasticity = (demand_change_i[non_zero_indices] /
price_change_j[non_zero_indices]).mean()
            cross_price_elasticity.loc[crop_i, crop_j] = elasticity
        else:
            cross_price_elasticity.loc[crop_i, crop_j] = np.nan
    else:
        cross_price_elasticity.loc[crop_i, crop_j] = np.nan

# 保存交叉价格弹性矩阵
cross_price_elasticity.to_excel('E:/2024 国赛/交叉价格弹性矩阵.xlsx')

```

第三问

分析求解

```

import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# 1. 读取作物数据文件
file_path = 'E:/2024 国赛/merged_data_with_years.xlsx'
data = pd.read_excel(file_path)

# 2. 筛选出相关列进行分析：作物名称、单价、种植成本、亩产量、土地面积
relevant_columns = ['作物名称', '单价/(元/斤)', '种植成本/(元/亩)', '亩产量/斤', '土地面积']
data_filtered = data[relevant_columns]

# 3. 根据作物名称分组，计算平均值
data_grouped = data_filtered.groupby('作物名称').mean()

# 4. 计算余弦相似度（可替代性分析）

```

```

cosine_similarity_scores = pd.DataFrame(cosine_similarity(data_grouped),
                                         index=data_grouped.index,
                                         columns=data_grouped.index)

# 5. 计算交叉价格弹性（互补性分析，使用随机数据示例）
# 替换成实际交叉价格弹性矩阵
cross_price_elasticity_matrix = np.random.rand(len(data_grouped), len(data_grouped)) -
0.5 # 示例数据，值为 -0.5 到 0.5
cross_price_elasticity_scores = pd.DataFrame(cross_price_elasticity_matrix,
                                              index=data_grouped.index,
                                              columns=data_grouped.index)

# 6. 打印作物的可替代性（余弦相似度）
print("作物之间的可替代性（余弦相似度）得分：")
print(cosine_similarity_scores)

# 7. 打印作物的互补性（交叉价格弹性）
print("\n作物之间的互补性（交叉价格弹性）得分：")
print(cross_price_elasticity_scores)

# 8. 保存结果到文件
output_path = 'E:/2024 国赛/作物_替代性_互补性分析.xlsx'
with pd.ExcelWriter(output_path) as writer:
    cosine_similarity_scores.to_excel(writer, sheet_name='可替代性_余弦相似度')
    cross_price_elasticity_scores.to_excel(writer, sheet_name='互补性_交叉价格弹性')

print(f"\n 分析结果已保存到 {output_path}")

```

第三问

聚类分析

```

import pandas as pd
from sklearn.cluster import KMeans

# 读取余弦相似度数据
file_path = 'E:/2024 国赛/作物_替代性_互补性分析.xlsx'
cosine_similarity_scores = pd.read_excel(file_path, sheet_name='可替代性_余弦相似度',
                                         index_col=0)

# 使用 k=3 进行 KMeans 聚类
k = 3
cosine_similarity_matrix = cosine_similarity_scores.values
kmeans = KMeans(n_clusters=k, random_state=42)

```

```
kmeans.fit(cosine_similarity_matrix)

# 获取聚类标签
cluster_labels = kmeans.labels_

# 添加聚类标签到作物数据中
clustered_data = pd.DataFrame({
    '作物名称': cosine_similarity_scores.index,
    '聚类标签': cluster_labels
}).set_index('作物名称')

# 打印聚类结果
print("聚类结果：")
print(clustered_data)

# 保存聚类结果到 Excel 文件
output_path = 'E:/2024 国赛/作物聚类结果_k3.xlsx'
clustered_data.to_excel(output_path)
print(f"\n 聚类结果已保存到 {output_path}")
```